

数字逻辑设计

目录

1	数字系统与逻辑抽象	1
2	数制与编码	3
3	逻辑运算与布尔代数	8
4	标准表达式与不完全给定函数	12
5	卡诺图化简	15
6	传播延迟、竞争与冒险	19
7	组合逻辑电路的分析与设计	22
8	常用组合逻辑器件与可编程逻辑	27
9	Verilog 硬件描述与仿真	34
10	锁存器与边沿触发器	40
11	寄存器、计数器与节拍发生器	45
12	同步状态机的分析与设计	51
13	中规模芯片与序列信号发生器	59
14	实验资料、代码阅读与验证约定	66
15	Lab 1: 组合逻辑与加减多路复用器	68
16	Lab 2: 触发器与寄存器文件	70
17	Lab 3: 可调频率与方向的流水灯	72
18	Lab 4: 数码管扫描、消抖与十进制计数	74
19	Lab 5: UART 发送与字符串调度	77
20	Lab 6: UART 接收、显示与字符串匹配	79
21	历史 Lab 补充: 格雷计数、序列检测与记忆游戏	83
22	主要结论与术语	86

1 数字系统与逻辑抽象

1.1 数字信号、模拟信号与二值逻辑

数字逻辑设计 (Digital Logic Design) 研究以离散代码表示信息、以逻辑电路处理和存储信息的方法。模拟信号的幅值可在一定区间连续取值；数字信号用有限个规定的信号状态表示代码。实际导线上的电压仍是连续的物理量，逻辑分析只把处于规定电平区间的电压解释为逻辑值。

数字电路常采用二值逻辑：变量取 0 或 1。二进制适合由截止、导通等两种主要工作状态的开关器件实现。二极管具有单向导电性；双极型晶体管可利用截止和饱和状态实现开关；集成电路把器件和互连集中在芯片内。逻辑门在抽象层面完成逻辑运算，具体电气实现还须满足供电、门限及负载要求。

定义：电平与逻辑值

正逻辑把高电平解释为 1、低电平解释为 0；负逻辑采用相反的对应。逻辑值不是电压的数值本身。电平有效性应按器件规定的输入、输出电压范围判断，处于未定义区间的输入不能可靠地解释为 0 或 1。

数字信号的抗干扰能力来自电平区间与门限之间的余量，并非能够抵抗任意噪声。数字信号也不必仅在离散时刻变化；同步系统在时钟边沿采样、更新状态，而异步输入可以在任意时刻变化。

1.2 信息转换与分辨率

模拟物理量进入数字系统通常经历采样、量化和编码；模数转换 (A/D) 把模拟输入转换为数字代码，数模转换 (D/A) 把代码转换为相应的模拟输出。数字系统适于存储、复制、运算和可编程控制，但接口的转换误差仍会影响最终结果。

设转换位数为 n ，可表示 2^n 个等级。若 A/D 输入范围的宽度为 V_{FS} ，均匀量化模型中一个量化间隔为

$$\Delta V = \frac{V_{FS}}{2^n}.$$

课件把相对分辨率写为 $1/2^n$ 。若 D/A 的最大码对应输出 V_{max} ，并把最小码输出取为零，则从零码到满码有 $2^n - 1$ 个间隔，课件对应的相对步长为 $1/(2^n - 1)$ 。若另把标称满量程定义为 2^n 个步长，也会出现 $1/2^n$ 的 D/A 分辨率表达式。比较不同资料时须先统一满量程的定义。

例题：8 位均匀量化

A/D 输入范围为 0 至 5 V，位数为 8，求理想量化间隔。

解 $\Delta V = 5/256 \text{ V} \approx 19.53 \text{ mV}$ 。该结果描述理想分辨能力，不等于实际转换精度；偏移、非线性与噪声均可能造成额外误差。

1.3 组合逻辑、时序逻辑与设计层次

组合逻辑电路在输入稳定并经过传播延迟后，输出只由当前输入决定：

$$\mathbf{Z} = f(\mathbf{X}).$$

时序逻辑电路具有存储状态，输出和次态由输入及现态确定：

$$\mathbf{Q}^+ = g(\mathbf{Q}, \mathbf{X}), \quad \mathbf{Z} = h(\mathbf{Q}, \mathbf{X}).$$

其中 $\mathbf{X}$ 是输入向量， $\mathbf{Z}$ 是输出向量， $\mathbf{Q}$ 是现态， $\mathbf{Q}^+$ 是次态。上标 + 表示下一次状态更新后的值，并非算术加法。

设计可分为三个层次：系统设计划分功能模块并确定接口；逻辑设计利用门、触发器及功能部件实现模块；电路设计确定器件、电气连接与物理实现。硬件描述语言描述结构和行为，电子设计自动化工具辅助仿真、综合、布局布线和时序分析。



1.4 集成规模与可编程逻辑器件

小规模、中规模、大规模和超大规模集成电路分别常记作 SSI、MSI、LSI、VLSI。门数划分随技术和统计口径变化，课件给出的范围用于说明规模层次，不作为所有技术通用的严格边界。

可编程逻辑器件 (PLD) 通过配置改变逻辑功能。常见类别有可编程阵列逻辑 PAL、复杂可编程逻辑器件 CPLD，以及现场可编程门阵列 FPGA。它们把门、查找表、触发器和可编程互连等资源组合成指定电路。资源结构将在组合逻辑器件一章中讨论。

1.5 由分辨率指标确定转换位数

例题：按量化间隔选取 A/D 位数

依据课件 02 第 3 页的均匀量化模型。A/D 输入范围为 0 至 3.3 V，要求量化间隔不大于 2 mV。求满足要求的最少位数，并核对相邻两种位数。

解 输入范围宽度为 $V_{FS} = 3.3 \text{ V}$ ，指标须先统一单位： $2 \text{ mV} = 0.002 \text{ V}$ 。由 $\Delta V = V_{FS}/2^n$ 得

$$\frac{3.3}{2^n} \leq 0.002 \quad \Leftrightarrow \quad 2^n \geq 1650.$$

由于 $2^{10} = 1024 < 1650 \leq 2048 = 2^{11}$ ，最少须取 $n = 11$ 。逐一核对：

$$\Delta V_{10} = \frac{3.3}{1024} \text{ V} \approx 3.223 \text{ mV}, \quad \Delta V_{11} = \frac{3.3}{2048} \text{ V} \approx 1.611 \text{ mV}.$$

十位不满足要求，十一位满足要求。位数应向上取整，不能把对数结果四舍五入后直接选型；本题只约束理想量化间隔，没有给出实际转换误差指标。

2 数制与编码

2.1 位权、基数与数值表示

按位计数制中，每一位数符的意义由其位置和基数共同决定。设基数为 $r \geq 2$ ，整数部分有 p 位，小数部分有 n 位，则

公式：按位展开

$$(d_{p-1} \cdots d_0.d_{-1} \cdots d_{-n})_r = \sum_{i=-n}^{p-1} d_i r^i, \quad 0 \leq d_i < r.$$

r^i 是第 i 位的权， d_i 是该位数符的数值。二、八、十、十六进制的基数分别为 2, 8, 10, 16。

十六进制数符 A, B, C, D, E, F 分别表示十进制 10 至 15。最高有效位 (MSB) 是位权最大的位，最低有效位 (LSB) 是位权最小的位。固定宽度整数有前导零时，这两个名称仍指指定字中的最高、最低位置。

2.2 十进制与其他进制的转换

整数转换采用除基取余、逆序排列。反复用 r 除整数，先得到的余数是最低位；商为零后，将余数倒序排列。依据为 $N = r[N/r] + (N \bmod r)$ 。

小数转换采用乘基取整、顺序排列。设当前小数为 x ，计算 $rx = d + x'$ ，其中 $d = [rx]$ 、 $0 \leq x' < 1$ ； d 是下一位数符，对 x' 重复操作。余下小数为零时结束；若不结束，按所需位数截断或舍入。

例题：十进制数转换为二进制

把 $(87.4375)_{10}$ 转换为二进制。

解 整数部分的连续商为 43, 21, 10, 5, 2, 1, 0，对应余数为 1, 1, 1, 0, 1, 0, 1，逆序得到 1010111。小数部分为

$0.4375 \times 2 = 0.875$	取0,
$0.875 \times 2 = 1.75$	取1,
$0.75 \times 2 = 1.5$	取1,
$0.5 \times 2 = 1.0$	取1.

故 $(87.4375)_{10} = (1010111.0111)_2$ 。

有限十进制小数不一定是有限二进制小数。约分后的分母仅含因子 2 时，才能用有限二进制小数精确表示；例如 $1/10$ 的二进制展开无限循环。保留 k 位二进制小数并截断时，绝对误差小于 2^{-k} 。

2.3 二、八、十六进制的分组转换

因为 $8 = 2^3$ 、 $16 = 2^4$ ，一位八进制数对应三位二进制，一位十六进制数对应四位二进制。以小数点为边界，整数向左、小数向右分组；整数最左端和小数最右端可补零，不能在其他位置随意补零。

$$(10111000.1101)_2 = (270.64)_8 = (B8.D)_{16}.$$

八进制小数 0.64 对应 0.110100，末尾零不改变值。分组仅改变表示方式，不改变数值。

2.4 二进制编码、BCD 与有权码

编码给对象分配代码。数制规定代码的数值解释，码制规定代码与对象的对应。 n 位二进制有 2^n 个不同码字，给 N 个对象分配互不相同的等长码字须满足 $2^n \geq N$ 。

二—十进制编码（BCD，Binary-Coded Decimal）用四位二进制代码表示一位十进制数码。**8421 BCD** 码的四位权值为 8,4,2,1；仅使用 0000 至 1001，其余六个码字不是有效十进制数码。

$$(59)_{10} \leftrightarrow (0101\ 1001)_{\text{BCD}}, \quad (59)_{10} = (111011)_2.$$

BCD 的分组应按十进制数码处理，不能把整串 BCD 当作普通二进制数。

有权码的数值等于各位权值之和，但仅给权值不一定能唯一确定码表。例如 2421 码中 0101 与 1011 的位权和都为 5，必须约定选用哪一个码字。下表采用课件的非括号方案。

十进制	8421	2421	4221	5421	余 3 码
0	0000	0000	0000	0000	0011
1	0001	0001	0001	0001	0100
2	0010	0010	0010	0010	0101
3	0011	0011	0011	0011	0110
4	0100	0100	0110	0100	0111
5	0101	1011	1001	1000	1000
6	0110	1100	1100	1001	1001
7	0111	1101	1101	1010	1010
8	1000	1110	1110	1011	1011
9	1001	1111	1111	1100	1100

余 3 码（Excess-3）把每个十进制数码 d 编为四位二进制的 $d + 3$ ，有效码范围为 0011 至 1100。它是无权码；四个位不能直接按固定权相加得到原十进制数码。表中的 2421 码及余 3 码具有自补性质：某数码代码逐位取反后，对应 $9 - d$ 。

2.5 典型格雷码及相互转换

格雷码 (Gray Code) 使相邻码字只相差一位。课程使用二进制反射格雷码；完整码表的首尾也仅相差一位。其编号顺序与普通二进制不同。

方法：二进制与典型格雷码互换

设二进制为 $b_{n-1} \cdots b_0$ ，格雷码为 $g_{n-1} \cdots g_0$ 。

$$g_{n-1} = b_{n-1}, \quad g_i = b_{i+1} \oplus b_i \quad (0 \leq i < n-1).$$

反向转换从最高位开始累积异或：

$$b_{n-1} = g_{n-1}, \quad b_i = b_{i+1} \oplus g_i.$$

$\oplus$ 表示异或，两位不同时结果为 1。

反射构造法从一位表 0,1 开始：旧表正序前加 0，旧表逆序前加 1，合并成下一位宽的码表。三位典型格雷码依次为

000, 001, 011, 010, 110, 111, 101, 100.

二进制 1011 对应格雷码 1110。格雷码改变一位的性质有助于减少连续位置变化时的混合码错误，也用于卡诺图排列；它不能保证任意跳变均只有一位变化。

例题：余 3 码转换为格雷码

给定两个十进制数码的余 3 码 1001 0101，求其数值对应的典型格雷码。

解 两组码分别表示 $9-3=6$ 与 $5-3=2$ ，故数值为 62。若整数采用八位二进制， $62 = 00111110_2$ ，对应八位格雷码 00100001。若按每个数码分别编码，则 $6 = 0110_2 \mapsto 0101$ 、 $2 = 0010_2 \mapsto 0011$ ，结果为 0101 0011。码制转换题须明确整值还是逐数码转换以及输出位宽。

2.6 字符编码

ASCII 把控制字符、数字、拉丁字母等映射为字符代码。标准 ASCII 是七位编码，共 128 个代码，常存放在最高位为零的八位字节中；课件中“8 位”应理解为常见存储宽度。字符 0、A、a 的代码分别为十进制 48, 65, 97，即十六进制 30, 41, 61。字符 0 的代码不等于数值零。

GB2312、GBK、GB18030 是中文字符编码方案；Unicode 为字符分配统一码点，UTF-8 等编码形式规定码点如何转换为字节序列。字符、码点和存储字节须区分。ASCII 位数依据 RFC 20 核对。

2.7 有限位小数与格雷码的计算

例题：无限二进制小数的八位截断
按课件 02 第 7 页的乘基取整方法，将 $(0.1)_{10}$ 转为二进制小数，保留八位并截断。求所表示的十进制值及截断误差。

解 每次乘 2，整数部分成为下一位，余下小数继续计算：

步数	当前小数乘 2	取出的位	剩余小数
1	$0.1 \times 2 = 0.2$	0	0.2
2	$0.2 \times 2 = 0.4$	0	0.4
3	$0.4 \times 2 = 0.8$	0	0.8
4	$0.8 \times 2 = 1.6$	1	0.6
5	$0.6 \times 2 = 1.2$	1	0.2
6	$0.2 \times 2 = 0.4$	0	0.4
7	$0.4 \times 2 = 0.8$	0	0.8
8	$0.8 \times 2 = 1.6$	1	0.6

余下小数再次出现 0.2，后续位会循环，故不能得到有限的精确表示。八位截断结果及其数值为

$$(0.00011001)_2 = 2^{-4} + 2^{-5} + 2^{-8} = \frac{25}{256} = 0.09765625.$$

以“原数减截断值”定义误差，则

$$e = 0.1 - 0.09765625 = 0.00234375 < 2^{-8} = 0.00390625.$$

截断与舍入是不同规则；本题不依据第九位进位。给定位数后，仍须说明采用哪一种规则。

例题：九位典型格雷码的正向与反向转换
课件 02 第 17 页给出二进制字 101101101。求对应典型格雷码，再由所得格雷码恢复原二进制字。比较二进制数 255 与 256 之间的位变化。

解 最高位复制，其余位由原二进制相邻两位异或得到。反向转换用已恢复的较高位与当前格雷位异或，不能把两个相邻格雷位直接异或。

位下标	8	7	6	5	4	3	2	1	0
二进制 b_i	1	0	1	1	0	1	1	0	1
格雷码 g_i	1	1	1	0	1	1	0	1	1
恢复的 b_i	1	0	1	1	0	1	1	0	1

例如 $g_7 = b_8 \oplus b_7 = 1 \oplus 0 = 1$ ；恢复时 $b_7 = b_8 \oplus g_7 = 1 \oplus 1 = 0$ ，接着 $b_6 = b_7 \oplus g_6 = 0 \oplus 1 = 1$ 。最终得到 $g = 111011011$ ，恢复结果与原字相同。

采用固定九位宽度，两个连续数及其典型格雷码为

$$255: 011111111_2 \rightarrow 010000000_{\text{Gray}},$$

$$256: 100000000_2 \rightarrow 110000000_{\text{Gray}}.$$

二进制字的九位都变化，格雷码只有最高位变化。单个位变化的性质针对码表中连续编号，不意味着任意两个码字只差一位。

3 逻辑运算与布尔代数

3.1 基本运算与复合运算

逻辑变量只取 0, 1。本讲义以 AB 或 $A \cdot B$ 表示与，以 $A + B$ 表示或，以 A' 表示非。运算优先顺序为非、与、或；复合表达式用括号明确范围。逻辑加法不等于算术加法： $1 + 1 = 1$ 。

定义：常用逻辑运算

与在全部输入为 1 时输出 1；或在至少一个输入为 1 时输出 1；非使 0, 1 互换。与非、或非分别对与、或的结果取反：

$$F_{\text{NAND}} = (AB)', \quad F_{\text{NOR}} = (A + B)'.$$

两输入异或与同或为

$$A \oplus B = A'B + AB', \quad A \odot B = AB + A'B' = (A \oplus B)'.$$

A	B	AB	$A + B$	$(AB)'$	$(A + B)'$	$A \oplus B$	$A \odot B$
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

与或非门可表示 $F = (AB + CD)'$ 。常用器件如 74LS08（与）、74LS32（或）、74LS04（非）、74LS00（与非）、74LS02（或非）、74LS86（异或）；门数、扇入和输出结构应根据具体器件资料确定。

异或具有交换律和结合律， $A \oplus 0 = A$ 、 $A \oplus 1 = A'$ 、 $A \oplus A = 0$ 。多个变量异或等于 1，当且仅当其中 1 的个数为奇数；“恰好一个输入为 1”仅是两输入异或的等价说法。

3.2 逻辑函数、真值表与逻辑图

布尔表达式由变量、常量和逻辑运算组成。真值表列出全部输入组合与相应输出，含 n 个不同输入的完全给定函数有 2^n 行。两个表达式在全部组合上的输出相同，才表示同一个逻辑函数。

逻辑图以门符号和连线表示结构。输入端或输出端的小圆圈表示反相或低电平有效；国际标准矩形符号常用“&”表示与、“ ≥ 1 ”表示或、“ $= 1$ ”表示两输入异或。识图时须同时读取门功能、反相标记和输入顺序。

例题：从文字建立逻辑函数

报警开关打开且门未关，或者已过下午六点且窗未关时报警。设 A 表示开关打开， B 表示门关闭， C 表示已过六点， D 表示窗关闭，报警输出为 Z 。

解 按两个条件的或连接, 得到 $Z = AB' + CD'$ 。若实际要求开关控制所有报警, 则应改为 $Z = A(B' + CD')$; 条件作用范围必须由需求确定。

3.3 基本定律

以下 A, B, C 可代表变量, 也可代表完整逻辑表达式。

名称	恒等式
同一律与零一律	$A + 0 = A, A \cdot 1 = A, A + 1 = 1, A \cdot 0 = 0$
互补律	$A + A' = 1, AA' = 0$
幂等律	$A + A = A, AA = A$
交换律	$A + B = B + A, AB = BA$
结合律	$(A + B) + C = A + (B + C), (AB)C = A(BC)$
分配律	$A(B + C) = AB + AC;$ $A + BC = (A + B)(A + C)$
双重否定	$(A')' = A$
德摩根定律	$(A + B)' = A'B', (AB)' = A' + B'$
吸收律	$A + AB = A, A(A + B) = A$
合并律	$AB + AB' = A, (A + B)(A + B') = A$
扩展吸收	$A + A'B = A + B, A(A' + B) = AB$
共识定理	$AB + A'C + BC = AB + A'C;$ $(A + B)(A' + C)(B + C) = (A + B)(A' + C)$

第二分配律是布尔代数特有的关系, 不能按普通代数删去。例如

$$\begin{aligned}(A + B)(A + C) &= A + AC + AB + BC && \text{分配及幂等律} \\ &= A + BC && \text{吸收律.}\end{aligned}$$

共识定理的依据是 $BC = BC(A + A') = ABC + A'BC$, 这两项分别被 AB 、 $A'C$ 吸收。

3.4 代入、反演与对偶规则

代入规则允许把恒等式中的同一变量一致替换成同一逻辑表达式。反演规则求表达式的反: 把与、或互换, 0, 1 互换, 每个变量取反, 并保持原运算层次。它来自逐层使用德摩根定律。

$$F = A(B + C') + D \quad \Rightarrow \quad F' = (A' + B'C)D'.$$

对偶规则把与、或互换, 把 0, 1 互换, 但变量及变量上的反号不变。保持原括号所限定的层次, 例如

$$F = A(B + C') \quad \Rightarrow \quad F^D = A + BC'.$$

若 $F = G$ 为恒等式, 则 $F^D = G^D$ 也是恒等式。对偶函数一般不等于反函数; 两者关系为

$$F^D(\mathbf{X}) = [F(\mathbf{X}')]'$$

其中 $\mathbf{X}'$ 表示每一个输入都取反。

3.5 代数化简与门电路变换

代数化简通常依次合并互补项、吸收冗余项、提取公因子, 再根据需要变成与或式或或与式。化简目标要说明: 两级与或实现可先比较乘积项数, 再比较总文字数; 在限定门类型、扇入、负载或速度时, 物理代价可能另有最优解。

例题: 化简并转换为与非结构

化简 $F = A'BC + AB'C' + AB'C + ABC' + ABC$, 并写成与非—与非形式。

解

$$\begin{aligned} F &= A'BC + AB'(C' + C) + AB(C' + C) \\ &= A'BC + AB' + AB \\ &= A'BC + A(B' + B) = A'BC + A \\ &= A + BC \end{aligned}$$

扩展吸收律。

因此 $F = [A'(BC)']'$ 。用反相器得到 A' 、用与非门得到 $(BC)'$, 最后用与非门合成。单独用与非门可将同一信号接两个输入实现反相; 或非门也具有构造任意布尔函数的能力。

有限扇入门应按结构分解。与、或具有结合律, 可用二输入树分组; 与非、或非不能不加反相就连续串接。例如 $((AB)'C)' = AB + C'$, 不等于 $(ABC)'$ 。化简后的逻辑等价还须结合延迟、冒险及可用器件判断。

3.6 代数化简与反演对偶的分步计算

例题: 五变量表达式中的吸收关系

2024 秋期末回忆版, 简答题 2: 用代数法化简

$$Y = AB' + BD + CDE + A'D,$$

要求写出每一步的依据。

解 将含 D 的第二、第四项合并, 再使用扩展吸收律 $X + X'Z = X + Z$:

$$\begin{aligned} Y &= AB' + D(B + A') + CDE \\ &= AB' + D(AB')' + CDE \\ &= AB' + D + CDE \\ &= AB' + D \end{aligned}$$

提取公因子 D ,
德摩根律 $(AB')' = A' + B$,
令 $X = AB'$, $Z = D$,
吸收律 $D + DCE = D$.

最终 $Y = AB' + D$, 变量 C, E 不再影响输出。还可按 D 分情况核对: $D = 0$ 时原式只剩 AB' ; $D = 1$ 时, 若 $AB' = 0$, 必有 $A' = 1$ 或 $B = 1$, 原式仍输出 1。因此化简后的函数覆盖全部 32 种输入组合。

例题: 含括号表达式的反函数与对偶式

取课件 03-1 第 22 页的表达式 $F = [A(C + D)]' + BE$ 。分别求反函数 F' 与对偶式 F^D , 说明两者是否相同。

解 反演须对整个函数取反, 保留作用范围, 逐层使用德摩根律:

$$\begin{aligned} F' &= ([A(C + D)]' + BE)' \\ &= A(C + D)(BE)' \\ &= A(C + D)(B' + E'). \end{aligned}$$

对偶变换只互换与、或及常量 0, 1, 已有反号保持原作用范围。 $A(C + D)$ 的对偶为 $A + CD$, BE 的对偶为 $B + E$, 因此

$$F^D = [A + CD]'(B + E).$$

取 $A = 0, B = 1, C = D = E = 0$, 有 $F' = 0$, 而 $F^D = 1$, 故两者不相同。反函数描述输出互补; 对偶式来自运算规则的替换, 不能用“给整个表达式加反号”求得。

4 标准表达式与不完全给定函数

4.1 最小项、最大项与编号

固定变量顺序 A, B, C , 把 A 作为最高位、 C 作为最低位, 输入 ABC 的二进制数值作为编号。

定义：最小项与最大项

n 个变量的最小项 m_i 是包含全部变量且每个变量只出现一次的乘积项, 仅在编号 i 的输入上为 1。编号位为 1 写原变量, 为 0 写反变量。

最大项 M_i 是包含全部变量且每个变量只出现一次的和项, 仅在编号 i 的输入上为 0。编号位为 0 写原变量, 为 1 写反变量。始终有 $M_i = m'_i$ 。

例如 $5 = (101)_2$, 故 $m_5 = AB'C$ 、 $M_5 = A' + B + C'$ 。 AB 对三变量函数而言不是最小项, $A + C$ 也不是最大项。

$$\sum_{i=0}^{2^n-1} m_i = 1, \quad \prod_{i=0}^{2^n-1} M_i = 0.$$

这里的求和、求积分别指逻辑或、逻辑与。不同最小项满足 $m_i m_j = 0$, 不同最大项满足 $M_i + M_j = 1$ ($i \neq j$)。

4.2 由真值表写标准式

标准与或式是输出为 1 的全部最小项之和, 标准或与式是输出为 0 的全部最大项之积。设 $\mathcal{O}$ 为输出 1 的编号集, $\mathcal{Z}$ 为输出 0 的编号集, 则

$$F = \sum_{i \in \mathcal{O}} m_i = \prod_{i \in \mathcal{Z}} M_i.$$

同一完全给定函数的两组编号互为补集, 不能直接照抄同一组编号。

例题：三人多数表决函数的标准式

三个输入 A, B, C 中至少两个为 1 时输出 $F = 1$ 。

解 输出 1 的组合是 011, 101, 110, 111, 输出 0 的组合是 000, 001, 010, 100, 因此

$$\begin{aligned} F &= \sum m(3, 5, 6, 7) = A'BC + AB'C + ABC' + ABC, \\ &= \prod M(0, 1, 2, 4) \\ &= (A + B + C)(A + B + C')(A + B' + C)(A' + B + C). \end{aligned}$$

化简后为 $F = AB + AC + BC$ 。标准式在固定变量集合和顺序下唯一, 最简式未必唯一。

4.3 一般表达式展开为标准式

与项缺少变量 X 时, 利用 $X + X' = 1$ 补全:

$$AB = AB(C + C') = ABC + ABC'.$$

和项缺少变量 X 时，利用第二分配律补全：

$$A + B = (A + B + C)(A + B + C').$$

随后删除重复最小项或最大项。求反时，原函数的成零编号变成反函数的成一编号；例如 $F = \sum m(3, 5, 6, 7)$ 时， $F' = \sum m(0, 1, 2, 4)$ 。

4.4 无关项的含义与使用条件

不完全给定函数只对部分输入指定输出。未要求输出的组合称为无关项 (don't-care terms)，常记为 d 或 $\times$ 。原因可能是输入在约束下不出现，或该输入下的输出不被使用。

$$F = \sum m(0, 3, 7) + \sum d(1, 6)$$

表示 0, 3, 7 必须输出 1，1, 6 可以自由选取，余下 2, 4, 5 必须输出 0。这是一种列表记号，并非给无关项添加普通固定真值。

无关项可在化简时选为 0 或 1，但不必全部使用。化简结果只须在规定输入上满足要求；当实际系统可能出现原以为不存在的输入时，原无关项假设必须重新审查。

注意：三种不同的记号

真值表中的 $\times$ 表示设计规格不关心该值；Verilog 的 x 表示四态仿真中的未知； z 表示高阻。三个含义不同，不能相互替代。状态机中未使用的编码还关系到非法状态恢复，不能只把它们当作化简资源。

4.5 展开定理与函数降维

对任意输入 X ，令 $F_1 = F|_{X=1}$ 、 $F_0 = F|_{X=0}$ ，称为关于 X 的余因子。逐个考察 $X = 0, 1$ ，可得香农展开：

定理：两种展开式

$$F = XF_1 + X'F_0 = (X' + F_1)(X + F_0).$$

余因子仅含其余变量。第一式直接对应二选一数据选择器；第二式可由第一式或逐值验证得到。

例如 $F = AB + A'C$ 关于 A 展开就是 $F = AB + A'C$ ，两个数据端分别接 C 与 B 。多次展开把复杂函数分成较少变量的函数，也解释了卡诺图降维与多路选择器实现的关系。

4.6 乘积项展开与最小项去重

例题：把一般与或式写成标准式

2024 秋期末回忆版，简答题 1：将 $Y = AB + C'D'$ 表示成最小项之和。固定变量次序为 A, B, C, D ， A 为编号最高位；同时写出最大项之积。

解 AB 缺少 C, D , 用 $C + C' = 1, D + D' = 1$ 补齐; $C'D'$ 缺少 A, B , 同理补齐:

$$\begin{aligned}
 AB &= AB(C + C')(D + D') \\
 &= ABC'D' + ABC'D + ABCD' + ABCD \\
 &= m_{12} + m_{13} + m_{14} + m_{15}, \\
 C'D' &= (A + A')(B + B')C'D' \\
 &= A'B'C'D' + A'BC'D' + AB'C'D' + ABC'D' \\
 &= m_0 + m_4 + m_8 + m_{12}.
 \end{aligned}$$

m_{12} 在两项展开中重复, 依据 $m_{12} + m_{12} = m_{12}$ 只保留一次。逻辑相加不是把重复项的系数变为 2。因此

$$Y = \sum m(0, 4, 8, 12, 13, 14, 15).$$

在全部编号 0 至 15 中, 其余编号输出为零, 故

$$Y = \prod M(1, 2, 3, 5, 6, 7, 9, 10, 11).$$

例如输入 1100 的编号为 12, 虽同时满足 AB 与 $C'D'$, 输出仍为逻辑 1。编号必须按指定变量次序解释, 不能把某个乘积项中省略的变量直接当作零。

5 卡诺图化简

5.1 格雷排列与逻辑相邻

卡诺图 (Karnaugh Map) 把真值表的 2^n 个输入组合排入网格, 相邻格应只有一个变量取值不同。二位坐标按 00, 01, 11, 10 排列。三变量可用 A 作行、 BC 作列; 四变量可用 AB 作行、 CD 作列。

$AB \backslash CD$	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

同一行左右边界、同一列上下边界也相邻, 四个角能构成四格圈; 对角接触不表示相邻。五变量图宜拆成某变量为 0, 1 的两张四变量图; 两张图的对应格相邻。若把五变量排为一张长图, 还须识别不表现为几何接触的逻辑相邻。

5.2 填写卡诺图

由真值表逐格填写; 由 $\sum m$ 在所列编号填 1; 由 $\prod M$ 在所列编号填 0。不完全给定函数先标 $\times$, 再标规定的 0, 1。一般乘积项可直接覆盖它规定的变量取值, 其余变量任意。例如四变量函数中的 AB' 覆盖 1000, 1001, 1010, 1011。

按逐格逻辑运算, 也可得到 F' 、 $F + G$ 、 FG 、 $F \oplus G$ 的图。这种运算须使用相同变量次序和坐标。

5.3 读取与或式与或与式

方法: 卡诺圈的规则

与或化简圈 1, 或与化简圈 0。圈必须构成变量自由变化的矩形组, 格数为 1, 2, 4, 8, ...; 允许跨边界、允许重叠。圈 1 时不能含规定的 0, 圈 0 时不能含规定的 1, 无关项可按需加入。必须覆盖全部规定目标格。

n 变量图中 2^k 格合法圈消去 k 个变化变量, 剩余 $n - k$ 个变量。圈 1 时, 不变值为 1 写原变量, 为 0 写反变量, 形成乘积项; 将各项逻辑相加。圈 0 时, 不变值为 0 写原变量, 为 1 写反变量, 形成和项; 将各项相乘。

先尽量扩大合法圈, 再选覆盖全部目标格的圈集。不能只凭“圈尽量大”就断言最简: 应找出只能由某一最大圈覆盖的目标格, 保留相应必要圈, 再比较余下覆盖方案的项数和文字数。不能被更大圈包含的乘积项称为质蕴含项; 含有只能由它覆盖的 1 格的称为必要质蕴含项。

例题：边界相邻与四格消去

化简 $F(A, B, C) = \sum m(0, 2, 4, 6, 7)$ ，同时写出最简与或式与或与式。

解 卡诺图如下。

$A \backslash BC$	00	01	11	10
0	1	0	0	1
1	1	0	1	1

实线：跨边界四格 C' 虚线：底行二格 AB

左右两端的四个 1 组成跨边界圈，其中只有 $C = 0$ 不变，得 C' ；底行 m_6, m_7 组成二格圈，得 AB 。因此

$$F = C' + AB.$$

圈零时， m_1, m_3 得和项 $A + C'$ ， m_1, m_5 得 $B + C'$ ，故 $F = (A + C')(B + C')$ 。用第二分配律可验证两式等价。

5.4 带无关项的化简与代码转换

例题：8421 BCD 数码不小于 5 的判定

输入 $ABCD$ 为有效 8421 BCD 码， A 为最高位。数码不小于 5 时输出 $F = 1$ 。

解 成一编号为 5, 6, 7, 8, 9，无关编号为 10 至 15。利用无关项得到 $F = A + BC + BD = A + B(C + D)$ 。有效输入中的 4 = 0100 输出 0，5 = 0101 输出 1。若输入允许全部四位二进制数，10 至 15 是否输出 1 必须由新需求确定。

8421 BCD 码 $WXYZ$ 转为余 3 码 $ABCD$ 时，每个输出分别化简，输入 10 至 15 作无关项，可取

$$\begin{aligned} A &= W + XY + XZ, \\ B &= X'Y + X'Z + XY'Z', \\ C &= YZ + Y'Z', \\ D &= Z'. \end{aligned}$$

这些公式仅承诺十个有效数码。也可用四位加法器执行 $WXYZ + 0011$ ，但两种实现的非法输入行为未必相同。

5.5 多变量图与代价判断

五变量函数以 $A = 0, 1$ 分层时，可把两层对应的合法圈合并，消去 A ；只出现在一层的圈须保留 A 或 A' 。若只分别化简两层再相加，可能遗漏跨层合并。

卡诺图适合少变量的手工化简。最简与或式和最简或与式以不同结构为目标，代价未必相同。多输出电路还可共享乘积项，逐输出独立最简未必给出整体最少门数。最后实现须考虑扇入、输出极性和延迟；无冒险实现也可能需要额外冗余项。

5.6 四变量与五变量卡诺图例题

例题：十二个最小项的四变量化简

2024 秋期末回忆版，简答题 3：求

$$Y(A, B, C, D) = \sum m(0, 1, 2, 3, 4, 6, 8, 9, 10, 11, 12, 14)$$

的最简与或式， A 为最高位。

解 行取 AB 、列取 CD ，均按 00, 01, 11, 10 排列。逐格按编号填写：

$AB \backslash CD$	00	01	11	10
00	1	1	1	1
01	1	0	0	1
11	1	0	0	1
10	1	1	1	1

第一行与最后一行跨边界相邻，构成八格圈

$$\{0, 1, 2, 3, 8, 9, 10, 11\} \rightarrow B'.$$

其中 A, C, D 均遍历 0, 1，只有 $B = 0$ 不变。第一列与最后一列也跨边界相邻，构成另一八格圈

$$\{0, 2, 4, 6, 8, 10, 12, 14\} \rightarrow D'.$$

两个圈允许重叠，且共同覆盖全部十二个 1，故

$$Y = B' + D'.$$

最简性可由零格判断：仅当 $B = D = 1$ 时为零，故 $Y = (BD)'$ 。单个乘积项不能同时覆盖 m_1 与 m_4 而又避开零格 m_5 ，因此至少需要两项；所得两项均只有一个文字，不能再减少。

例题：五变量图的跨层合并与孤立格

课件 03-3 第 38 页：

$$F(A, B, C, D, E) = \sum m(0, 1, 4, 5, 6, 11, 12, 14, 16, 20, 22, 28, 30, 31).$$

以 $A = 0, 1$ 分层，用卡诺图化简。 A, B, C, D, E 依次为编号从高到低的五位。

解 两层均以 BC 为行、 DE 为列，按格雷顺序排列；右层编号比对应左层大 16。

$A = 0$	$DE = 00$	01	11	10	$A = 1$	$DE = 00$	01	11	10
$BC = 00$	1	1	0	0	$BC = 00$	1	0	0	0
01	1	1	0	1	01	1	0	0	1
11	1	0	0	1	11	1	0	1	1
10	0	0	1	0	10	0	0	0	0

先找只能由一个最大圈覆盖的格，再把对应圈扩大。选取的圈与保留变量为

圈内编号	乘积项	消去的变量
0, 1, 4, 5	$A'B'D'$	C, E 遍历，保留 $A = B = D = 0$
0, 4, 16, 20	$B'D'E'$	跨层消去 A ，并消去 C
4, 6, 12, 14, 20, 22, 28, 30	CE'	两层对应四格合并；消去 A, B, D
11	$A'BC'DE$	逻辑相邻的五格全为零，不能扩大
30, 31	$ABCD$	仅 E 变化

所以

$$F = A'B'D' + B'D'E' + CE' + A'BC'DE + ABCD.$$

格 $m_1, m_{16}, m_6, m_{11}, m_{31}$ 分别只能由表中对应最大圈覆盖，五项都不能省略。跨层合并须使用对应格；不能因为两层分别有 1 就把不对应的格合为一圈。每一圈都只覆盖给定的成一编号，圈集的并集恰为题设集合。

6 传播延迟、竞争与冒险

6.1 传播延迟与路径差异

输入改变后，逻辑门输出需要一段时间才进入对应的稳定电平。传播延迟 t_{pLH} 表示输出低到高的延迟， t_{pHL} 表示输出高到低的延迟；两者可不同，也受负载影响。多条信号路径汇合时，即使输入源同时改变，到达汇合点的时刻也可能不同。

竞争指多条路径的信号变化存在先后差异并可能产生相互影响；若输出因此出现逻辑上不应有的短脉冲或额外跳变，称为冒险或险象（hazard）。静态逻辑等价不保证瞬态波形等价。

6.2 静态、动态与功能冒险

类型	理想行为	可能的实际行为
静态 1 冒险	输出持续为 1	短暂跌为 0 再回到 1
静态 0 冒险	输出持续为 0	短暂升为 1 再回到 0
动态冒险	输出应只跳变一次	跳变多次后才稳定
功能冒险	多个输入由初值变为终值	中间输入组合造成错误输出

静态和动态冒险常由单个输入变化沿不同路径传播引起；功能冒险涉及多个输入变化。发生冒险的实际脉冲宽度取决于路径延迟，逻辑分析通常判断“存在产生冒险的可能性”。

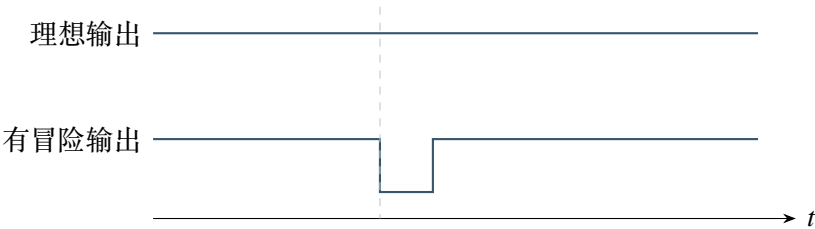
例题：静态 1 冒险及冗余项

两级与或电路 $F = AB + A'C$ ，当 $B = C = 1$ 时，输入 A 改变。

解 稳态逻辑为 $F = A + A' = 1$ 。若原先导通的乘积项先变为 0，另一项尚未变为 1，输出可能出现低脉冲。加入共识项 BC ：

$$F_{\text{safe}} = AB + A'C + BC.$$

由共识定理，它与原函数稳态等价； $B = C = 1$ 时， BC 持续提供 1，覆盖两项交接的空隙。



6.3 代数判别与卡诺图判别

代数判别可寻找在固定其他输入后变成 $X + X'$ 或 XX' 的不同信号路径。仅看到某变量及其反变量出现，不足以直接断言有冒险，还须分析实现结构和固定条件。

对两级与或电路，若两个逻辑相邻的 1 格没有被同一个乘积项圈共同覆盖，则单输入变化时存在静态 1 冒险的可能。补入跨越这两个格的圈，可消除该相邻转换的静态 1 冒险。对两级或与电路，同样检查相邻 0 格是否被同一和项的零圈覆盖。

$$F = (A + B)(A' + C)$$

在 $B = C = 0$ 时可能出现静态 0 冒险，加入和项 $B + C$ 得

$$F_{\text{safe}} = (A + B)(A' + C)(B + C).$$

上述判据的前提是规定的两级结构、单输入变化和合法电平；不能据此保证任意多级电路或多输入跳变均无冒险。

6.4 功能冒险与工程处理

若 F 在初值、终值上均为 1，但两个输入变化所经过的某个中间组合上为 0，真实电路可能出现功能冒险。例如 $F = A'BC + AB' + AC$ ，固定 $A = 1$ ，输入 BC 从 00 变成 11：若 B 先变，则经过 10，输出可能从 1 跌为 0 再回到 1；若 C 先变，经过 01，输出保持 1。这种问题不能通过加入与原函数等价的冗余项改变规定成零组合的输出。

课件介绍增加冗余项、滤波电容和封锁/选通脉冲等方法。冗余项适于上述静态冒险；滤波属于电气处理，会改变负载与边沿，不能在逻辑模型中任意添加；选通信号应在电路充分稳定后采样。同步系统通常通过寄存器采样、时序约束和稳定的时钟使能控制避免把毛刺当作有效事件。组合译码信号直接用作时钟或异步清零时，对毛刺尤其敏感。

6.5 由路径到达时间分析静态 0 冒险

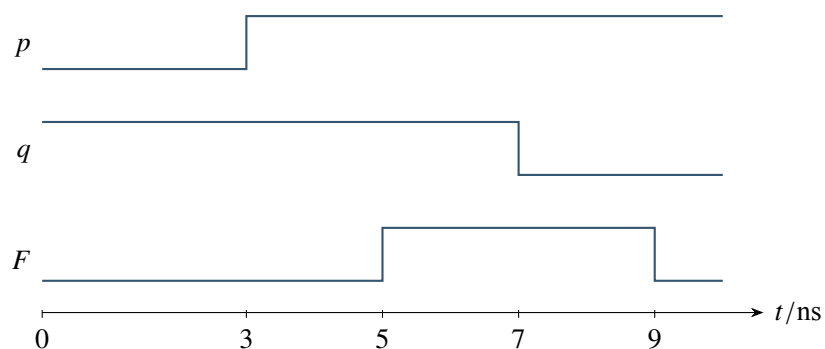
例题：错误高脉冲的形成与消除

依据课件 03-4 第 7 页的两级或与电路。设 $F = (A + B)(A' + C)$ ， $B = C = 0$ ， A 从 0 变为 1。两和项 $p = A + B$ 、 $q = A' + C$ 在末级与门输入处分别于 3 ns、7 ns 完成变化；末级与门采用 2 ns 的纯传输延迟模型。求输出可能出现的错误脉冲，并给出逻辑等价的消除办法。

解 稳态下 $p = A, q = A'$ ，故理想输出始终为 $AA' = 0$ 。但传播途中，两个节点不是在同一时刻改变：

时间区间	p	q	末级门的逻辑输入乘积 pq
$t < 3 \text{ ns}$	0	1	0
$3 \leq t < 7 \text{ ns}$	1	1	1
$t \geq 7 \text{ ns}$	1	0	0

中间四纳秒出现 $p = q = 1$ 。按题定纯传输模型，两个跳变均延后 2 ns，因此 F 在 $5 \leq t < 9 \text{ ns}$ 为 1，形成宽度 4 ns 的错误高脉冲。这是静态 0 冒险。



或与结构可加入冗余和项 $B + C$ ：

$$F_{\text{safe}} = (A + B)(A' + C)(B + C).$$

依据共识定理的对偶式，加入该项不改变稳定逻辑功能。当 $B = C = 0$ 时，这个和项持续为零，使末级输出保持零，不再依靠两条 A 路径恰好同时切换。实际门可能具有惯性延迟，短脉冲是否传播还取决于器件；本题的脉冲时刻仅适用于给定的纯传输模型。

7 组合逻辑电路的分析与设计

7.1 分析步骤与设计步骤

组合电路分析从已知电路出发：为中间节点命名，逐级写表达式，代入得到输出函数，化简并列真值表，最后说明实际功能。设计从需求出发，方向相反。

方法：组合逻辑设计的完整步骤

1. 规定输入、输出的意义、位宽、有效电平和合法输入集合。
2. 把需求写成真值表；只有确定不使用的组合才标无关项。
3. 写标准表达式或填写卡诺图，按所需结构化简。
4. 按门类型、扇入、输出极性与共享项转换实现形式，画逻辑图或编写 HDL。
5. 核对全部合法组合，再检查延迟、冒险、使能和负载条件。

规模、速度、功耗、价格和可靠性可能相互制约。不同实现可具有相同真值表而物理性能不同。

7.2 表决器、裁判电路与操作码形成器

三人多数表决输出为 $F = AB + AC + BC$ 。若主裁判 A 与副裁判 B, C 按课件规则控制红、绿灯，红灯为 R 、绿灯为 G ，则

$$R = A + BC, \quad G = A(B + C).$$

主裁判单独同意，或只有两名副裁判同意时， $R = 1, G = 0$ ；主裁判与至少一名副裁判同意时，两灯均亮。其余情况两灯均灭，且 $G = 1$ 必然有 $R = 1$ 。

操作码形成器设 A, B, C 分别为乘、加、减键，只有单键按下才有效；约定无键按下输出 00，乘、加、减输出分别为 01, 10, 11。以 F_2F_1 为输出，得到

$$F_2 = B + C, \quad F_1 = A + C.$$

若规定同时按键必须输出错误标志，不能继续把多键组合当作无关项，须另设有效性输出或重新定义操作码。

7.3 多输出函数：两位数的平方

例题：平方电路

无符号两位输入 $X = x_1x_0$ ，四位输出 $Y = y_3y_2y_1y_0 = X^2$ 。

解 $X = 0, 1, 2, 3$ 时输出分别为 0000, 0001, 0100, 1001，逐列得到

$$y_3 = x_1x_0, \quad y_2 = x_1x'_0, \quad y_1 = 0, \quad y_0 = x_0.$$

这里 y_2 必须包含 x'_0 ，因为 $X = 3$ 的平方为 9，其 2^2 位为 0。多输出设计应分别核对每一列，而非由某个输出的化简结果推测其他输出。

7.4 半加器、全加器与进位

半加器相加两个一位数 a_i, b_i ，输出本位和 S_i 与进位 C_i ：

$$S_i = a_i \oplus b_i, \quad C_i = a_i b_i.$$

全加器还接受来自低位的进位 C_{i-1} 。根据三个一位输入的算术和，和位是奇偶性，进位是多数函数：

公式：全加器

$$S_i = a_i \oplus b_i \oplus C_{i-1},$$

$$C_i = a_i b_i + (a_i \oplus b_i) C_{i-1} = a_i b_i + a_i C_{i-1} + b_i C_{i-1}.$$

C_{-1} 表示最低位的外部输入进位，与课件编号一致。

a_i	b_i	C_{i-1}	S_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

四位数 $1011 + 1110$ 的五位结果为 11001 ；最低位和到最高位和构成 1001 ，末级进位为 1 。输出宽度若只有四位，最高进位会被截去。

多个全加器串行进位时，高位必须等待低位进位，最坏延迟随位数线性增长。设进位传递 $P_i = a_i \oplus b_i$ 、进位产生 $G_i = a_i b_i$ ，则 $C_i = G_i + P_i C_{i-1}$ 。逐层展开得

$$C_0 = G_0 + P_0 C_{-1},$$

$$C_1 = G_1 + P_1 G_0 + P_1 P_0 C_{-1},$$

$$C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{-1},$$

$$C_3 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 \\ + P_3 P_2 P_1 P_0 C_{-1}.$$

超前进位利用输入直接并行生成进位，以更多组合逻辑减少逐位等待。大位宽实现常分组处理，不能假设任意长乘积项可由无延迟、无限扇入的门实现。课件采用异或定义 P_i ，有些资料采用 $a_i + b_i$ ；两种进位约定不能未经说明混用和位公式。

7.5 全减器与借位

全减器计算 $a_i - b_i - B_{i-1}$, B_{i-1} 为低位借入, 输出差位 D_i 与向高位的借位 B_i 。用 B_i 区别于加法进位:

$$D_i = a_i \oplus b_i \oplus B_{i-1}, \quad B_i = a'_i b_i + a'_i B_{i-1} + b_i B_{i-1}.$$

当 $a_i = 0$ 时, 只要 b_i 或借入为 1 就须借位; 当 $a_i = 1$ 时, 只有二者均为 1 才须借位。这也得到

$$B_i = a'_i (b_i \oplus B_{i-1}) + b_i B_{i-1}.$$

$1110_2 - 1011_2 = 0011_2$, 最终借位为零。加法与减法的和位、差位均为三输入异或, 但进位与借位函数不同。

7.6 三态门与总线

三态缓冲器在输出 0,1 以外, 还可输出高阻态 Z 。高电平使能 E 的非反相缓冲器在 $E = 1$ 时输出输入 A , 在 $E = 0$ 时输出 Z 。高阻近似电气断开, 不是第三种普通布尔真值。

多个输出通过三态门连接同一总线时, 任一时刻通常只能有一个驱动器有效。两个有效驱动器分别输出 0,1 会造成总线争用; 所有驱动器禁用时, 总线不等于自动输出零, 是否有确定电平取决于上拉、下拉或保持电路。双向总线的方向与使能应共同控制。

例题: 选择能被 5 整除的 BCD 数码

输入 $X = x_3 x_2 x_1 x_0$ 为 8421 BCD 码, 只有数码 0,5 时允许其数据驱动输出总线。

解 使能函数的成一编号为 0,5, 无关编号为 10 至 15, 可取

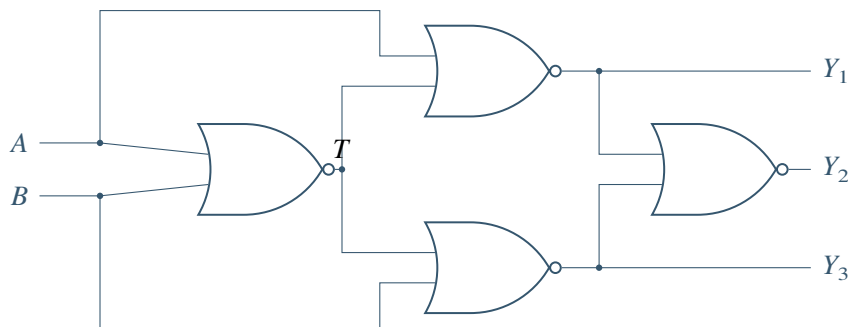
$$E = x_2 x'_1 x_0 + x'_3 x'_2 x'_1 x'_0.$$

各数据位通过共同受 E 控制的三态门传送。 $E = 0$ 时输出为高阻, 而非四位零码; 零数码被选中时则主动输出 0000。这两种状态必须区分。

7.7 从门电路逐级求函数

例题: 四个或非门组成的比较电路

2024 秋期末回忆版, 简答题 6。分析下图, 写出 Y_1, Y_2, Y_3 的最简与或式及逻辑功能。输入 A, B 均为一位无符号数。



解 门输出端的小圆圈表示取反。先给第一级输出命名 T ，再写后继函数；导线分支上的实心点表示电气连接。

$$T = (A + B)',$$

$$Y_1 = (A + T)' = A'T' = A'(A + B) = A'B,$$

$$Y_3 = (B + T)' = B'T' = B'(A + B) = AB',$$

$$Y_2 = (Y_1 + Y_3)' = (A'B + AB')' = AB + A'B'.$$

最后一式可由异或取反得到，也可按德摩根律展开后吸收。列出真值表核对输出的含义：

A	B	Y_1	Y_2	Y_3	比较结果
0	0	0	1	0	相等
0	1	1	0	0	$A < B$
1	0	0	0	1	$A > B$
1	1	0	1	0	相等

这是一个一位数值比较器： Y_1 指示小于， Y_2 指示相等， Y_3 指示大于，稳定时三者恰有一个为 1。若只看门的数量或图形外观，无法判断功能；须从连接关系求出函数。反相小圆圈属于门符号，不能当作导线上的分支连接点。

7.8 文字条件到控制电路的设计

例题：缆车启动条件

2024 秋期末回忆版，设计题 1。两辆缆车的方向分别为 A, B ：1 表示上行，0 表示下行。门状态为 C ：1 表示关闭。要求两车一上一下且门关闭才允许启动，输出 $Y = 1$ 表示允许。求函数并画逻辑图。

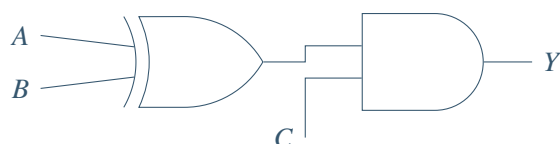
解 “一上一下”只对应 $AB = 01$ 或 10 ，不是“至少一车上行”。再与关门条件相与，得到

$$Y = (A'B + AB')C = A'BC + AB'C = (A \oplus B)C.$$

按全部八种组合列表，成一编号仅为 3,5：

A	B	C	Y	不允许时的原因
0	0	0	0	方向相同且门开
0	0	1	0	方向相同
0	1	0	0	门开
0	1	1	1	满足全部条件
1	0	0	0	门开
1	0	1	1	满足全部条件
1	1	0	0	方向相同且门开
1	1	1	0	方向相同

用一个异或门判断方向不同，再用与门加入关门条件：



若仅允许与、或、非门，就把异或展开为 $A'B + AB'$ ，再与 C 相与。验算 $AB = 11, C = 1$ 必须输出零；误写为 $(A + B)C$ 会在这一组合错误地允许启动。本题实现逻辑条件；实际设备还须由完整控制系统处理故障与执行过程，题目未要求状态存储。

8 常用组合逻辑器件与可编程逻辑

8.1 二进制译码器及使能极性

二进制译码器将 n 位代码转换为 2^n 条选择线, 启用时只有一条有效, 每一条对应一个最小项。输出高电平有效时 $Y_i = m_i$; 低电平有效时输出引脚电平 $y_i = m'_i = M_i$ 。

74LS138 为三线—八线译码器, 地址输入 CBA 中 C 为最高位、 A 为最低位。它要求 $G_1 = 1$ 、 $G_{2A} = G_{2B} = 0$ 才启用; 启用时被选中的输出为 0、其余为 1, 禁用时全部输出为 1。这里 G_{2A}, G_{2B} 指引脚电平, 即使器件图上的名称带横线, 仍须按低电平有效读表。该极性依据 TI 74LS138 数据手册核对。

例题：译码器实现全加器

把全加器输入 a, b, c 分别接到 74LS138 的 C, B, A , 并使器件启用。

解 和位 $S = \sum m(1, 2, 4, 7)$, 进位 $C_{out} = \sum m(3, 5, 6, 7)$ 。由于 $y_i = m'_i$, 用与非门合成

$$S = (y_1 y_2 y_4 y_7)', \quad C_{out} = (y_3 y_5 y_6 y_7)'.$$

全减器的差位同样使用 1, 2, 4, 7, 借位使用 1, 2, 3, 7。低有效输出不能直接用普通或门合成成一最小项。

8.2 译码扩展与地址译码

两片 74LS138 共用低三位地址 CBA , 第四位 D 分别启用低组、高组, 可构成四线—十六线译码器。令低组 $G_1 = 1, G_{2A} = D, G_{2B} = 0$; 高组 $G_1 = D, G_{2A} = G_{2B} = 0$ 。 $D = 0$ 时只有低组启用, $D = 1$ 时只有高组启用。

地址译码用高位地址确定片选, 低位地址确定器件内部单元。未参加片选比较的地址位能自由变化; 若本应参与的高位被遗漏, 会出现地址镜像。

例题：16 位地址空间分配

八个存储芯片各有 13 位内部地址, 以 $A_{15}A_{14}A_{13}$ 译码选择芯片。求第 k 片 ($k = 0, \dots, 7$) 的地址范围。

解 每片有 $2^{13} = 8192$ 个地址, 范围为

$$[k \cdot 2^{13}, (k+1)2^{13} - 1].$$

从第一片到第八片依次为 0000–1FFF、2000–3FFF、4000–5FFF、6000–7FFF、8000–9FFF、A000–BFFF、C000–DFFF、E000–FFFF (均为十六进制)。

若使能固定 $A_{15} \dots A_{12} = 0001$, $CBA = A_{11}A_{10}A_9$, 低九位自由, 则整体范围为 1000–1FFF; 选择编号 2 的外设范围为 1400–15FF。求范围须从实际连线读出所有固定条件, 不能只数地址线。

8.3 显示译码与七段数码管

显示译码器把输入代码转换为点亮字形所需的段选信号。七段从顶部开始依次标为 a (上)、 b (右上)、 c (右下)、 d (下)、 e (左下)、 f (左上)、 g (中)。共阴极通常高电平点亮，共阳极通常低电平点亮；还需限流及合适驱动能力。

数码	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1
7	1	1	1	0	0	0	0
8	1	1	1	1	1	1	1
9	1	1	1	1	0	1	1

上表为常用共阴极字形，1 表示点亮。设 $ABCD$ 是 8421 BCD 输入，利用非法码作无关项，前三段可取

$$a = A + C + BD + B'D', \quad b = B' + CD + C'D', \quad c = B + C' + D.$$

其他段按对应列分别化简。若非法输入须熄灭，必须把相应输出指定为零，而不能继续作无关项。

8.4 多路选择器与函数实现

多路选择器 (MUX, 也称多路复用器、数据选择器) 按 n 位选择信号，在 2^n 路数据中选一路输出。启用时

$$Y = \sum_{k=0}^{2^n-1} m_k(\mathbf{S}) I_k.$$

$\mathbf{S}$ 是选择向量， m_k 是其最小项， I_k 是第 k 路输入。二选一为 $Y = S'I_0 + SI_1$ ，可直接实现香农展开。

若函数输入数等于选择位数，数据端按真值表接 0, 1；若输入数少于选择位数，剩余选择端接确定常量，并按实际编号接数据；若输入数多于选择位数，数据端接剩余输入的余因子。

例题：四选一实现三人多数函数

选择端取 AB ，用四选一 MUX 实现 $F = AB + AC + BC$ 。

解 固定 AB , 依次得到 $I_0 = 0, I_1 = C, I_2 = C, I_3 = 1$ 。这比八选一直接接八个常量少用一个选择输入。

四变量函数降成三变量图时, 每格是关于余下变量 D 的函数。若 $F|_{D=0}, F|_{D=1}$ 分别为 0, 0, 0, 1, 1, 0, 1, 1, 该数据端分别接 0, $D, D', 1$ 。再少一个选择位时, 数据端须实现两变量余因子, 而非只能接上述四种值。

74LS151 为八选一, 低有效选通端 $G = 0$ 时, 同相输出 $Y = I_k$, 反相输出 $W = I'_k$; $G = 1$ 时 $Y = 0, W = 1$, 两者均不是高阻。这依据 TI 74LS151 数据手册。74LS153 包含两个四选一单元, 公共选择端、独立低有效使能, 禁用输出为零。

扩展为三十二选一的一种直接方案是四个八选一作为第一级, 共用低三位选择, 再用一个四选一按高两位选其输出。也可用译码器分别使能四片 74LS151, 但须按禁用输出极性添加汇合门: 同相输出可用或门汇合, 不能直接把推挽输出短接。

多路分配器 (DEMUX) 把一路数据送到选择的一个输出, 其余输出为规定的非有效值。高有效结构可写 $Y_k = m_k(S)D$ 。MUX 控制数据源, DEMUX 控制数据目的地; 译码器提供选择条件。

8.5 普通编码器、优先编码器与有效标志

普通编码器在 2^n 路输入中仅一路有效的前提下, 输出该路的 n 位编号。四线—二线高有效编码器的输出为 $B_1 = I_2 + I_3, B_0 = I_1 + I_3$ 。多个输入同时有效时, 这两式不再保证正确对象编号。

优先编码器允许多路请求同时有效, 按指定优先级输出最高优先级的编号。若 $I_3 > I_2 > I_1 > I_0$, 可取

$$B_1 = I_3 + I_2, \quad B_0 = I_3 + I'_2 I_1, \quad V = I_3 + I_2 + I_1 + I_0.$$

V 是有效标志, 区分“没有请求”和“第 0 路有效”; 二者的编号都可能是 00。

课件的低有效十进制键盘以 $P_1, \dots, P_9$ 表示按键线, 最多一键有效, 零键或无键均编为 0000。对应 8421 码输出为

$$\begin{aligned} W &= (P_8 P_9)', & X &= (P_4 P_5 P_6 P_7)', \\ Y &= (P_2 P_3 P_6 P_7)', & Z &= (P_1 P_3 P_5 P_7 P_9)'. \end{aligned}$$

若须区分数字零与无键, 应另设按键有效信号。优先编码器记录的是优先规则, 不会自动记录哪个按键最先到达; 抢答功能还需状态存储。

8.6 奇偶校验

设数据位为 $A_{n-1}, \dots, A_0$, 异或和为 $P = \bigoplus_i A_i$ 。偶校验使包括校验位在内的总 1 数为偶数, 故校验位 $P_E = P$; 奇校验使总 1 数为奇数, 故 $P_O = P'$ 。

公式: 接收端检测

包括校验位的接收字为 $R_n \dots R_0$ 。偶校验错误标志为 $E_E = \bigoplus_{i=0}^n R_i$; 奇校验错误标志为 $E_O = (\bigoplus_{i=0}^n R_i)'$ 。标志为 1 表示不满足约定的奇偶性。

任意奇数位翻转都会改变奇偶性，任意偶数位翻转不会改变奇偶性。因此校验通过仅表示没有发现奇偶性错误，不能断言没有传输错误；奇偶校验不能定位错误或纠正错误。

74LS280 有九个输入，输出 ODD 在输入含奇数个 1 时为 1，EVEN 则是其反。用它生成八位数据的奇校验位，可把第九输入接 0，取 EVEN；检测时九位输入为数据与奇校验位，取 EVEN 作为错误标志。示例数据 10110010 含四个 1，偶校验位为 0，奇校验位为 1。

8.7 数值比较器及级联

一位比较器有三个互斥输出：相等 $E = A \odot B$ 、大于 $G = AB'$ 、小于 $L = A'B$ 。无符号多位数从高位开始比较，只有高位相等才比较低位。

令第 i 位 $e_i = a_i \odot b_i$ 、 $g_i = a_i b_i'$ 、 $l_i = a_i' b_i$ 。四位比较为

$$E = e_3 e_2 e_1 e_0,$$

$$G = g_3 + e_3 g_2 + e_3 e_2 g_1 + e_3 e_2 e_1 g_0,$$

$$L = l_3 + e_3 l_2 + e_3 e_2 l_1 + e_3 e_2 e_1 l_0.$$

74LS85 的级联输入传入低位组的比较结果；独立比较一组时按“相等、大于、小于”的顺序置为 1,0,0。串行级联把低位输出传到高位输入，最高组产生总结果。

并行分组时，各组先独立比较，再由组间逻辑从高组向低组决定结果。以四组为例，将 e_i, g_i, l_i 换为每组的结果，仍用上述形式。课件的 64 位实现把八组 8 位的 GT、LT 标志组成两个 8 位向量，再比较这两个向量：最高一个不相等的组在 GT、LT 中分别为 10 或 01，故第二级比较正确判定全字大小；若全部组相等，两向量均为零。该方法依赖组输出互斥及组序正确，不能把任意标志向量如此比较。

逻辑规模随位数增长；串行优先链的深度随位数或组数线性增加，树形分层可降低逻辑深度。比较有符号数时，必须使用相应的有符号解释，不能套用无符号高位比较规则。

8.8 ROM、乘积项阵列与 FPGA 查找表

只读存储器 (ROM) 在固定内容、异步读模型中可看作一个 n 输入、 b 输出的组合逻辑：输入是地址，存储的每一行是该地址的输出。组织为 $2^n \times b$ ，总容量为 $2^n b$ 位。实际同步读存储器可能带输出寄存器，不能一概视为纯组合电路。

例题：ROM 实现四位乘四位无符号乘法

两个四位数 X, Y 输入，ROM 输出乘积。

解 地址可取 $\{X, Y\}$ ，共八位、256 种组合。最大乘积 $15 \times 15 = 225 = (11100001)_2$ ，需要八位输出，故用 256×8 ROM，容量 2048 位。每个地址存放对应乘积。程序可生成表内容，再用独立算术结果核对。

输出极性可控的二线—四线译码器有两个地址输入和一个极性控制输入，共三位；输出四位，故可用 8×4 ROM。高有效与低有效两套译码表各占四行。

在乘积项结构中, PROM 常对应固定与阵列、可编程或阵列; PLA 的与、或阵列均可编程; PAL 通常与阵列可编程、或阵列固定; GAL 可再编程并提供可配置宏单元。CPLD 通常组合多个逻辑块与互连, 具体实现依器件而定。

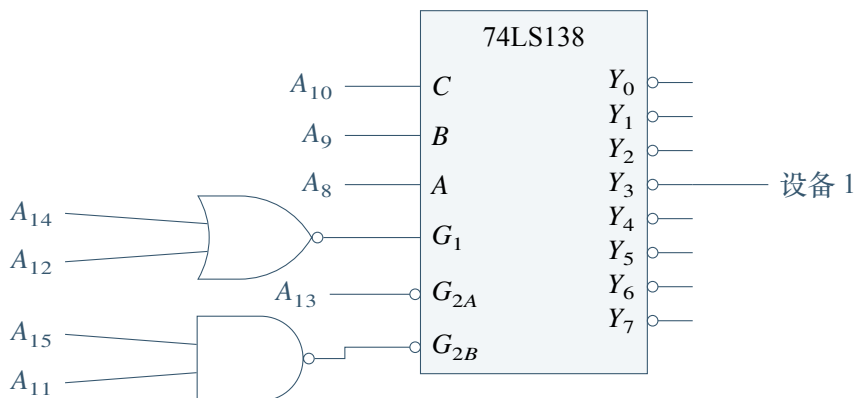
查找表 (LUT) 对 k 个输入存储 2^k 个输出真值, 可实现任意 k 输入的一位布尔函数。四输入与门的 16 个表项只有地址 1111 对应 1, 其余为 0。LUT 表内容可由配置存储位保存, 不能把所有 FPGA 一概描述为同一种存储技术。

FPGA 含 LUT、触发器、选择器、进位链与可编程互连, 还可能含块存储、DSP、输入输出块和时钟网络。可配置逻辑块 CLB 及 slice 的具体构成依器件系列而定; 课件中的 7 系列例子说明一种架构, 不是所有 FPGA 的统一规格。实现功能仍受资源数量、接口和时序限制。

8.9 复杂使能条件的地址译码

例题: 由使能门与译码编号求地址范围

2024 秋期末回忆版, 简答题 4。地址为 $A_{15} \cdots A_0$, 外设内部接低八位 $A_7 \cdots A_0$ 。电路连接见图: $G_1 = (A_{14} + A_{12})'$, $G_{2A} = A_{13}$, $G_{2B} = (A_{15}A_{11})'$; 设备 1 接低有效输出 Y_3 。题面明确规定译码器 A 端为高位、C 端为低位。求全地址译码范围及设备 1 范围, 并与通常 74LS138 编号约定比较。



解 先求启用条件。 $G_1 = 1$ 要求 $A_{14} = A_{12} = 0$; 两个低有效使能输入必须为零, 故 $A_{13} = 0$ 且 $A_{15} = A_{11} = 1$ 。所以

$$A_{15}A_{14}A_{13}A_{12}A_{11} = 10001.$$

剩余 $A_{10} \cdots A_0$ 全可变化, 整个电路的连续地址范围为

$$1000\ 1000\ 0000\ 0000_2 \text{ 至 } 1000\ 1111\ 1111\ 1111_2 \Rightarrow 8800 \sim 8FFF_{16}.$$

按本题文字规定, 译码编号的三位是 $ABC = A_8A_9A_{10}$ 。Y₃ 选中时 $A_8A_9A_{10} = 011$, 地址字中的实际高到低顺序则为 $A_{10}A_9A_8 = 110$ 。设备 1 因此占

$$8E00 \sim 8EFF_{16}, \quad 2^8 = 256 \text{ 个地址}.$$

通常 74LS138 数据手册及本讲义前文按 CBA 编号, A 为最低位。若按真实器件约定, Y_3 要求 $A_{10}A_9A_8 = 011$, 设备范围改为 $8B00 \sim 8BFF_{16}$, 整体范围仍为 $8800 \sim 8FFF_{16}$ 。回忆版题面与通常约定存在冲突, 应先注明采用哪一个约定, 不能混用两者得出唯一答案。

8.10 数据选择、优先编码与校验的完整计算

例题：八选一实现四变量函数

按课件 04-2 的数据选择器方法改编。用一个启用的八选一 MUX 实现

$$F(A, B, C, D) = \sum m(0, 1, 2, 5, 7, 8, 9, 10, 12, 14),$$

取 ABC 为从高到低的选择字, 求全部数据端 I_0 至 I_7 的连接。

解 固定 $ABC = k$, 分别检查编号 $2k$ 、 $2k+1$ 是否在成一集合中, 即得到 $D = 0, 1$ 时的两个输出。再按 $00 \rightarrow 0, 01 \rightarrow D, 10 \rightarrow D', 11 \rightarrow 1$ 翻译为余因子:

ABC	最小项编号	$D = 0$	$D = 1$	数据端
000	0, 1	1	1	$I_0 = 1$
001	2, 3	1	0	$I_1 = D'$
010	4, 5	0	1	$I_2 = D$
011	6, 7	0	1	$I_3 = D$
100	8, 9	1	1	$I_4 = 1$
101	10, 11	1	0	$I_5 = D'$
110	12, 13	1	0	$I_6 = D'$
111	14, 15	1	0	$I_7 = D'$

只须一个反相器提供 D' 。若使用 74LS151, 逻辑选择字 ABC 须按本讲义前文的器件位序接入: 器件 C, B, A 端分别接函数变量 A, B, C , 使能端接低电平, 取同相输出。输入 $ABCD = 1101$ 选 $I_6 = D' = 0$, 与编号 13 不在成一集合中一致; 不能按函数变量字母直接接同名字引脚而忽略位权。

例题：同时有效的请求与编码有效标志

按课件 04-2 的优先编码方法改编。八路高有效请求 $I_7 \cdots I_0$, 下标越大优先级越高, 输出编号 $B_2B_1B_0$ 和有效标志 V 。求请求字 01001000 的输出, 并解释为什么普通编码器不适用。

解 请求字中 $I_6 = I_3 = 1$, 应只编码第 6 路。为逐层屏蔽低优先级请求, 定义

$$H_7 = I_7, \quad H_k = I_k \prod_{j=k+1}^7 I'_j \quad (0 \leq k < 7).$$

$H_k = 1$ 表示第 k 路为本次获选请求，至多一个 H_k 为 1。输出为

$$\begin{aligned} B_2 &= H_4 + H_5 + H_6 + H_7, \\ B_1 &= H_2 + H_3 + H_6 + H_7, \\ B_0 &= H_1 + H_3 + H_5 + H_7, \quad V = \sum_{k=0}^7 I_k. \end{aligned}$$

本题 $H_6 = 1$ ，而 H_3 的乘积中含 $I'_6 = 0$ ，所以 $H_3 = 0$ ，得到 $(B_2 B_1 B_0, V) = (110, 1)$ 。若把普通编码器的输出直接写成对应请求的或， I_6 与 I_3 同时有效会得到错误的 111，表示根本没有请求的第 7 路。

边界情况也须区分：仅 $I_0 = 1$ 时输出 $(000, 1)$ ；没有请求时规定输出 $(000, 0)$ 。编号零不等于没有请求，因此有效标志不能省略。低有效芯片须按其功能表转换极性，不能直接照搬本题高有效式。

例题：奇偶校验的生成、检验与漏检

按课件 04-2 的奇偶校验方法改编。八位数据为 11010010，求偶校验位和奇校验位；以偶校验传输为例，比较一个数据位翻转与两个数据位翻转。

解 数据中有四个 1，异或和为 $P = 0$ ，所以偶校验位 $P_E = 0$ ，奇校验位 $P_O = 1$ 。把校验位附在数据后，两个传输字为

110100100（偶校验）， 110100101（奇校验）。

接收端偶校验错误标志是包括校验位的全部九位异或，而不是只检查八位数据：

接收情况	九位接收字	1 的总数	E_E
无翻转	110100100	4	0
仅最高数据位翻转	010100100	3	1
最高及最低数据位翻转	010100110	4	0

一个翻转被发现，两个翻转虽改变数据，却维持偶数个 1，因而漏检。校验位自身翻转也属于一位错误，可以检出。若用 74LS280 生成奇校验位，把第九输入置零并取 EVEN；接收奇校验字时把九位都输入，EVEN 为高才表示违反奇校验要求。

9 Verilog 硬件描述与仿真

9.1 描述层次、并行性与综合

硬件描述语言（HDL）描述电路结构、功能和时序。Verilog 的结构化描述实例化并连接子模块；数据流描述常用连续赋值表示信号关系；行为描述用过程块描述输入、状态和输出之间的规则。不同描述方式可以混合使用。

寄存器传输级（RTL）描述寄存器、组合运算和按时钟更新的数据流。综合把可实现的 HDL 转换成逻辑网表，再映射到目标器件。仿真用延迟、文件输出等语句描述测试过程，不一定对应硬件。HDL 中的可执行语句不等于软件指令流：各模块、连续赋值与过程块描述并行活动的电路，块内语句的仿真执行顺序才由赋值方式决定。

设计流程包括需求与规格、模块划分、RTL 编码、功能仿真、逻辑综合、实现及时序验证。静态时序分析检查路径是否满足时钟约束；功能仿真通过不代表已满足所有物理时序要求。

9.2 模块、接口与实例化

模块用 `module` 和 `endmodule` 包围。接口要说明方向和宽度，内部要声明连线或变量。标识符大小写敏感，普通标识符的首字符是字母或下划线，后续可含数字及 \$。单行注释为 `//`，多行注释为 `/* ... */`。

```
module half_adder(  
    input wire a, b,  
    output wire sum, carry  
);  
    assign sum = a ^ b;  
    assign carry = a & b;  
endmodule  
  
module full_adder(  
    input wire a, b, cin,  
    output wire sum, cout  
);  
    wire s1, c1, c2;  
    half_adder u0(.a(a), .b(b), .sum(s1), .carry(c1));  
    half_adder u1(.a(s1), .b(cin), .sum(sum), .carry(c2));  
    assign cout = c1 | c2;  
endmodule
```

实例化加入的是一个实际并行工作的功能部件。命名端口连接 `.a(a)` 的左侧是子模块端口名，右侧是父模块中的信号。连接时必须匹配位宽，避免把带反相含义的端口当作同相端口。

9.3 四态值、连线与过程变量

Verilog 基本仿真值为 0、1、x、z，分别表示低逻辑值、高逻辑值、未知、高阻。未初始化的过程变量常为未知；未驱动的正常连线常为高阻。

wire 表示由驱动源决定值的连线，通常由 assign 或子模块输出驱动。传统 Verilog 的 reg 表示在过程块中赋值的变量，并不保证综合成寄存器；完整赋值的组合过程仍对应组合逻辑。按边沿保持状态的过程通常综合成触发器，电平敏感且不完整赋值的过程可能综合成锁存器。

integer 常用于仿真循环计数，time 用于时间，real 用于实数仿真模型。parameter 为可覆盖参数，localparam 为模块内部固定常量。reg [7:0] mem [0:15] 声明 16 个八位元素，实际是否推断成存储器取决于读写描述和器件支持。

```
wire [7:0] bus;  
reg [3:0] q;  
parameter WIDTH = 8;  
localparam IDLE = 2'b00;
```

向量 [7:0] 的第 7 位为所声明范围的高索引位，bus[3:0] 是部分选择。输入由父模块信号驱动；传统 Verilog 中子模块输出连接到父模块的连线。显式声明能避免隐式一位连线造成拼写错误。

9.4 常量、位宽与运算符

有基数常量格式为“位宽、单引号、可选有符号标志、基数、数值”，例如 8'b10101100、8'hA2、4'd9、8'sd7。无位宽十进制常量通常至少 32 位，并涉及有符号规则；讲义示例尽量写出位宽。赋值到较窄信号时高位可能截断，较宽时的扩展取决于有无符号及表达式上下文。

类型	运算符	主要用途
算术	$+$ $-$ $*$ $/$ $\%$	整数运算，整数除法舍去小数部分
逐位	$\sim$ $\&$ $ $ $\wedge$ $\sim\wedge$	各对应位执行逻辑操作
逻辑	$!$ $\&\&$ $ $	将操作数按真假判断，结果为一位
归约	$\&a$ $ a$ $\wedge a$	把整个向量归约为一位
关系	$<$ $>$ $\leq$ $\geq$	大小比较，留意有符号性
相等	$==$ $!=$ $===$ $!==$	逻辑相等与四态严格相等
移位	$<<$ $>>$ $>>>$	左移、逻辑右移、有符号算术右移
条件	$cond ? a : b$	条件选择
拼接	$\{a,b\}$	合并固定位宽的向量

归约和逐位运算须区分，例如 $\wedge 4'b1011$ 为 1，而 $4'b1011 \wedge 4'b0011$ 为 $4'b1000$ 。
 $4'b0010 \& 4'b0100$ 为零，而逻辑与 $4'b0010 \&\& 4'b0100$ 为一，因为两个操作数都非零。

$==$ 、 $!=$ 遇未知位且不能决定相等关系时，可能返回 x ；存在已知确定的不等位时也可返回明确结果。 $===$ 、 $!==$ 将 x,z 作为可比较的状态，总返回 0 或 1，常用于测试平台。普通 $case$ 也按四态匹配。

逻辑右移 $>>$ 补零； $>>>$ 只有在左操作数按有符号量解释时才按符号扩展。移位结果位宽由表达式规则决定，不能依赖移位自动扩大位宽。拼接 $\{a,b\}$ 把 a 放高位、 b 放低位，重复拼接如 $\{4\{1'b0\}\}$ 为四位零。拼接元素应给出明确宽度。

注意：逻辑加法和算术加法

布尔式 $F = A + B$ 对应 Verilog 的 $A | B$ ，不是 $A + B$ 。后者执行二进制算术加法。保留进位可先把操作数扩成所需宽度，例如 `assign total = {1'b0,a} + {1'b0,b};`。

运算符优先级大致为一元运算、乘除余、加减、移位、关系、相等、逐位与、异或、逐位或、逻辑与、逻辑或、条件运算。复杂表达式优先用括号标明电路含义，不依赖记忆完成含糊解析。

9.5 连续赋值与组合过程

`assign` 位于过程块之外，对连线持续施加驱动；右侧变化会重新决定左侧值。它可描述导线、逻辑门或整个组合网络，不是只对应一根无功能导线。

```
module majority3(input wire a, b, c, output wire f);
    assign f = (a & b) | (a & c) | (b & c);
endmodule

module mux4(
    input wire [3:0] d,
    input wire [1:0] sel,
    output reg y
);
    always @(*) begin
        case (sel)
            2'b00: y = d[0];
            2'b01: y = d[1];
            2'b10: y = d[2];
            2'b11: y = d[3];
            default: y = 1'b0;
        endcase
    end
endmodule
```

组合过程采用 `always @(*)`，使读取的输入变化时重新求值；使用阻塞赋值 `=`，并保证每条路径对每个输出赋值。先给默认值、再按条件覆盖也是完整赋值的一种方法。

`if ... else if ... else` 可明确表达优先级。`case` 适于列举互斥码值。综合后的面积、延迟由逻辑函数、目标器件和优化决定，不能把所有 `if` 一概说成“资源少但慢”，也不能把所有 `case` 一概说成“资源多但快”。即使没有 `else`，若块开头已给默认值，仍可能是完整的组合描述；反之，写了 `default` 但遗漏某个输出的赋值，仍可能产生锁存器。

```
module priority8(
    input wire [7:0] req,
    output reg [2:0] code,
    output reg valid
);
    always @(*) begin
        code = 3'd0;
        valid = 1'b1;
        if (req[7]) code = 3'd7;
        else if (req[6]) code = 3'd6;
        else if (req[5]) code = 3'd5;
        else if (req[4]) code = 3'd4;
    end
endmodule
```



```

    else if (req[3]) code = 3'd3;
    else if (req[2]) code = 3'd2;
    else if (req[1]) code = 3'd1;
    else if (req[0]) code = 3'd0;
    else valid = 1'b0;
end
endmodule

```

该电路中最高下标请求优先；全零输入输出 code=0, valid=0。奇偶校验可用 assign odd = ^data; 与 assign even = ~(^data); 描述，而无须逐门书写。这些输出是检测输入字奇偶性的标志，作为校验位生成时须按总体奇偶要求选用。

9.6 阻塞与非阻塞赋值

阻塞赋值 = 在当前过程执行中更新左值，后续语句读到新值。非阻塞赋值 <= 先求右值并安排稍后的更新，使同一时钟事件下不同寄存器按旧状态计算次态。

```

always @(posedge clk) begin
    a <= d;
    b <= a;
    c <= b;
end

```

若时钟前 $a = 5, b = 3, c = 10, d = 2$ ，更新后 $a = 2, b = 5, c = 3$ ；三句改成按同样顺序的阻塞赋值则都成为 2。非阻塞赋值并非对所有代码顺序都无关：若同一变量在同一过程被多次赋值，后执行的赋值会影响最终更新。一个变量应由单一过程负责，避免多个过程争用。

采用“组合过程用阻塞、边沿时序过程用非阻塞”的约定。一个模块可以同时包含这两种过程，例如状态机的组合次态逻辑与时钟状态寄存器；不能把“模块中不能混用”理解为禁止这样的正常划分。

9.7 测试平台与自检示例

测试平台 (testbench) 一般没有外部端口，实例化待测模块，用 initial 产生激励、用延迟等待输出稳定，并检查结果。always 反复响应事件；initial 从仿真开始执行一次，其中仍可含循环或持续等待。固定边界的 for 可用于可综合逻辑的展开，仿真中的时间控制则是不同用途。

```

`timescale 1ns/1ps
module tb_full_adder;
    reg a, b, cin;
    wire sum, cout;
    reg [1:0] expected;
    integer k, errors;
    full_adder dut(.a(a), .b(b), .cin(cin),

```



```
        .sum(sum), .cout(cout));  
  
initial begin  
    errors = 0;  
    for (k = 0; k < 8; k = k + 1) begin  
        {a,b,cin} = k[2:0];  
        expected = {1'b0,a} + {1'b0,b} + {1'b0,cin};  
        #1;  
        if ({cout,sum} !== expected) begin  
            errors = errors + 1;  
            $display("Mismatch: input=%b%b%b", a,b,cin);  
        end  
    end  
    $display("errors=%0d", errors);  
    $finish;  
end  
endmodule
```

该测试枚举八个输入组合；正确设计的预期结果为 errors=0。!== 能把未知输出也判为不通过。#1 和 \$display、\$finish 属于仿真控制，不应照搬进待综合的普通组合功能模块。

时序测试还须产生时钟与复位，避免在采样边沿同时改变激励，并在非阻塞更新完成后检查输出。关键测试包括复位、保持、所有状态转移、计数边界、装载优先级、非法状态恢复；随机测试可补充，不能替代未覆盖的必需条件。

10 锁存器与边沿触发器

10.1 现态、次态与特征方程

存储元件通过反馈维持状态。记 Q 为现态, Q^+ 为次态, Q' 为当前输出的补。特征方程给定输入和现态, 求次态; 激励表 (驱动表) 从要求的现态、次态反求输入条件。分析电路用前者, 综合设计常用后者。

10.2 基本 SR 锁存器与门控 D 锁存器

两个交叉耦合或非门构成高有效 SR 锁存器。S 为置位输入, R 为复位输入: $S = R = 0$ 保持; $S = 1, R = 0$ 置一; $S = 0, R = 1$ 清零; $S = R = 1$ 时两个输出均为零, 不满足互补要求, 禁止作为正常输入。

公式: 高有效 SR 锁存器

$$Q^+ = S + R'Q, \quad SR = 0.$$

约束是公式正常使用的前提。从禁止输入同时释放到保持输入时, 最终状态可能取决于门延迟, 不能预先唯一确定。

交叉耦合与非门构成的低有效 SR 锁存器采用相反输入极性: 两输入均为 1 时保持, 置位输入为 0 时置一, 复位输入为 0 时清零, 两输入同时为 0 禁止。器件输入极性应从功能表读取。

SR 锁存器可用于双掷机械开关去抖: 开关离开一个触点后, 锁存器先保持, 接通另一个触点时才改变状态; 短暂返回同一触点不重复改变。此结论依赖输入不同时有效等电路条件, 不适用于任意单按钮接法。

门控 D 锁存器以使能 G 控制透明性: $G = 1$ 时输出跟随 D , $G = 0$ 时保持, 方程为

$$Q^+ = GD + G'Q.$$

透明期间输入变化可能多次传播到输出, 这就是课件所称的“空翻”。锁存器的透明性是其正常特性; 把它直接接成反馈计数结构时, 可能产生不期望的多次变化。74LS373 是八位 D 锁存器类器件, 须区分锁存使能和输出使能。

10.3 边沿触发的 SR、D、JK 与 T 触发器

边沿触发器只在规定的时钟有效边沿更新状态, 其余时间保持, 异步控制除外。时钟符号上的三角表示边沿触发, 三角前的反相圈表示下降沿触发。有效边沿到来并不意味着必然翻转: 是否置位、清零、保持或翻转取决于输入。

类型	特征方程	有效边沿时的功能
SR	$Q^+ = S + R'Q, SR = 0$	保持、置一、清零；有输入约束
D	$Q^+ = D$	采样并存储 D，无逻辑输入约束
JK	$Q^+ = JQ' + K'Q$	00 保持，01 清零，10 置一，11 翻转
T	$Q^+ = T \oplus Q$	$T = 0$ 保持， $T = 1$ 翻转
T'	$Q^+ = Q'$	每个有效边沿均翻转

T 触发器可由 JK 触发器令 $J = K = T$ 得到；T' 是令 $T = 1$ 的特例。 $Q^+ = D$ 并不表示 D 变化时 Q 立即变化，而是指有效采样边沿后的状态值。

10.4 激励表与类型转换

<i>Q</i>	<i>Q</i> ⁺	<i>S</i>	<i>R</i>	<i>J</i>	<i>K</i>	<i>D</i>	<i>T</i>
0	0	0	×	0	×	0	0
0	1	1	0	1	×	1	1
1	0	0	1	×	1	0	1
1	1	×	0	×	0	1	0

例如 JK 从 0 到 1 只须 $J = 1$ ， K 任意，因此激励表中的 × 可参与卡诺图化简。SR 表中的任意值仍必须与另一输入共同满足 $SR = 0$ 。

把一种触发器转换成另一功能时，先写目标次态，再利用现有触发器的特征方程或激励表决定驱动。用 D 实现 JK，令 $D = JQ' + K'Q$ ；用 JK 实现 D，令 $J = D, K = D'$ ；用 D 实现 T，令 $D = T \oplus Q$ 。转换改变的是输入组合逻辑，不能改变原触发器的边沿极性及异步端行为。

10.5 异步置位、清零与时钟使能

异步控制独立于时钟生效。设低有效预置和清零的引脚电平为 PRE_n, CLR_n ：前者为零、后者为一时置位；前者为一、后者为零时清零；两者为一时正常按时钟工作。两者同时为零通常属于禁止或特殊情况，须查具体器件功能表。

时钟使能 CE 只控制有效时钟沿是否装入新数据，不改变时钟本身：

$$Q^+ = CE \cdot D + CE'Q.$$

$CE = 0$ 时保持。用组合门把任意使能直接与时钟相与可能产生短脉冲或额外边沿；器件支持的时钟使能或专用时钟控制资源更适合实现规定的同步行为。

```
module dff_ce(input wire clk, rst_n, ce, d,
              output reg q);
    always @(posedge clk or negedge rst_n) begin
```

```

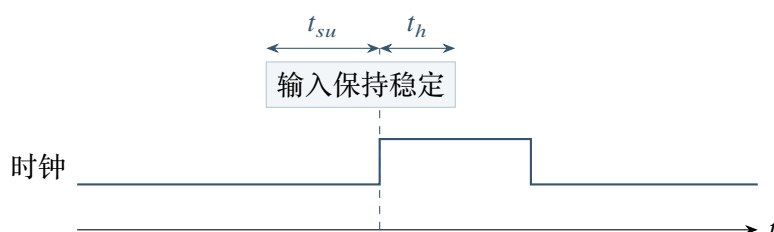
    if (!rst_n) q <= 1'b0;
    else if (ce) q <= d;
end
endmodule

```

省略时序过程的 else 表示不更新该寄存器，正好实现保持。课件时钟使能示例中的 else $Q \leq 0$ ；实现禁用时同步清零，与上述使能保持功能不同，不能直接作为相同器件使用。

10.6 建立、保持与复位释放

可靠采样要求输入在有效时钟沿前稳定至少建立时间 t_{su} ，沿后保持至少保持时间 t_h 。触发器输出通常在时钟到输出延迟 t_{CQ} 后改变。违反时序约束可能产生亚稳态，即输出暂时不能可靠解释为确定逻辑电平。



异步复位的释放也有恢复、移除等时序要求。常用“异步断言、同步释放”：两级触发器的异步端同时接受外部复位，释放后第一级装入 1、第二级再采样第一级，使本地复位按时钟释放。

```

reg [1:0] reset_pipe;
always @(posedge clk or negedge rst_async_n) begin
    if (!rst_async_n) reset_pipe <= 2'b00;
    else reset_pipe <= {reset_pipe[0], 1'b1};
end
wire rst_sync_n = reset_pipe[1];

```

两级结构降低亚稳态传播概率，不能宣称绝对消除。不同独立时钟域需要相应的释放同步。波形分析也须考虑真实传播延迟；课堂理想状态表只表示稳定状态。

10.7 按有效边沿分析波形

确定复位及初态，标出有效时钟边沿；按沿前输入与现态逐沿计算次态，再画出边沿之间的保持段。

例题：JK 波形的逐沿计算

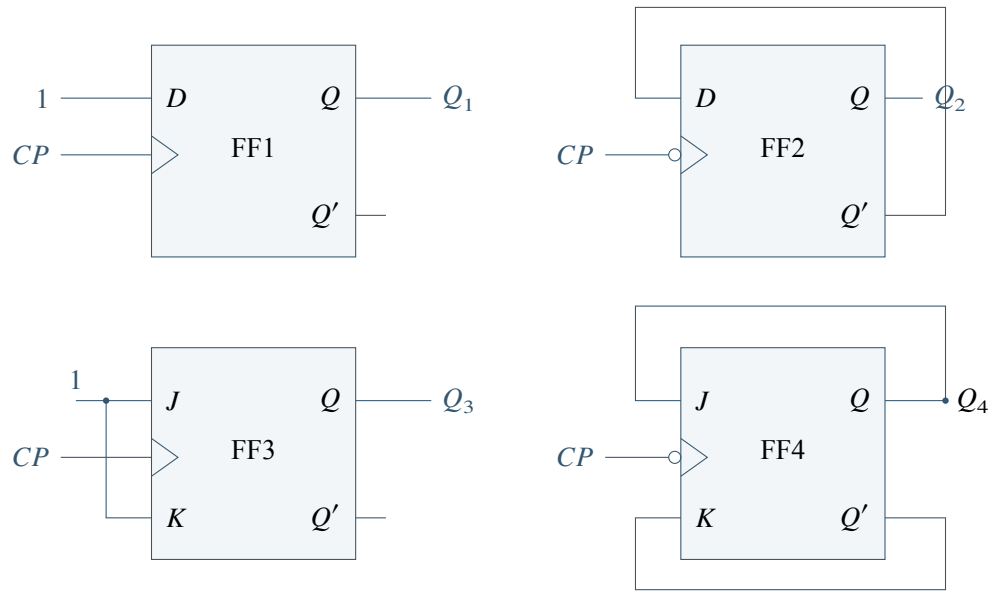
下降沿触发 JK 初态 $Q = 0$ ，连续五个下降沿处的 JK 依次为 01, 01, 11, 00, 00。

解 逐沿所得 Q 为 0, 0, 1, 1, 1，其中仅第三沿翻转。共用时钟时，各触发器同时按沿前现态计算次态。

10.8 从触发器接线逐沿画波形

例题：四种反馈接法与两种有效边沿

2024 秋期末回忆版，简答题 8。四个触发器的接线重绘如下。为明确波形起点，规定第一个待分析上升沿前 $Q_1 = Q_2 = Q_3 = Q_4 = 0$ ，忽略传播延迟，画出四个完整时钟周期内的输出。



解 先读时钟三角前是否有反相圈：FF1、FF3 为上升沿触发，FF2、FF4 为下降沿触发。 Q' 表示本触发器的反相输出；反馈并未把 Q' 变成时钟。

输出	接线与次态	有效边沿上的行为
Q_1	$D = 1, Q_1^+ = 1$	第一个上升沿置一，后续保持一
Q_2	$D = Q_2', Q_2^+ = Q_2'$	每个下降沿翻转
Q_3	$J = K = 1, Q_3^+ = Q_3'$	每个上升沿翻转
Q_4	$J = Q_4, K = Q_4'$	每个下降沿仍保持现态

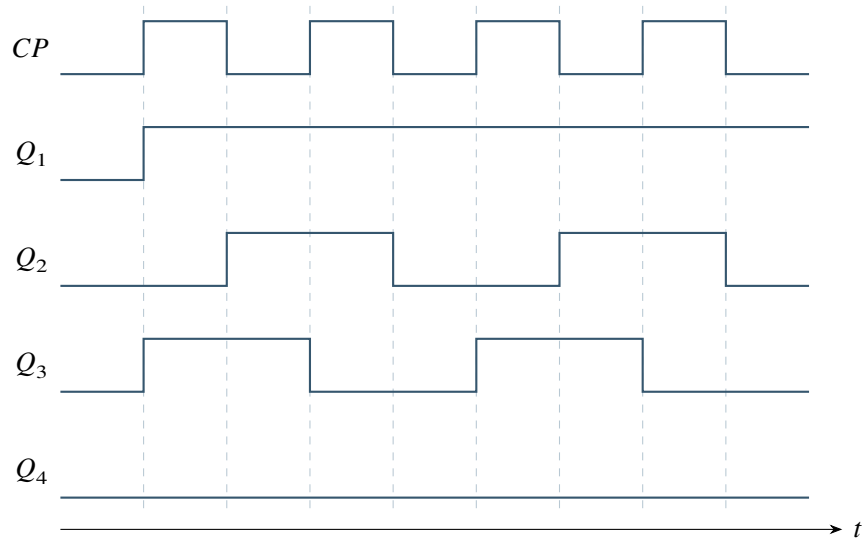
最后一项必须代入 JK 特征方程，而非凭“存在反馈”就判为翻转：

$$Q_4^+ = JQ_4' + K'Q_4 = Q_4Q_4' + Q_4Q_4 = Q_4.$$

若 $Q_4 = 0$ ，边沿前 $JK = 01$ ，清零后仍为零；若原为 1，则 $JK = 10$ ，置位后仍为一。因此本题 Q_4 始终为零。

按上升沿与下降沿交替列表；非本触发器的有效边沿处须保持，不更新其状态：

边沿之后	Q_1	Q_2	Q_3	Q_4
初态	0	0	0	0
第 1 上升沿	1	0	1	0
第 1 下降沿	1	1	1	0
第 2 上升沿	1	1	0	0
第 2 下降沿	1	0	0	0
第 3 上升沿	1	0	1	0
第 3 下降沿	1	1	1	0
第 4 上升沿	1	1	0	0
第 4 下降沿	1	0	0	0



波形由表中状态连成，输出仅在自身有效边沿跳变。理想图中省略 t_{CQ} ，实际输出应在边沿之后经过传播延迟才改变。回忆版图中的短起始线不能替代明确初态；若分析起点移到已有若干脉冲之后，须先确定该点状态，再重复同样方法。

11 寄存器、计数器与节拍发生器

11.1 基本寄存器与数据传送

寄存器用一组触发器存储有限位代码， n 位寄存器通常用 n 个一位触发器。写入、保持、复位改变或控制内部状态；读出把已存状态送到外部，通常不改变内部状态。

设并行输入 D 、装载使能 LD ，在有效时钟沿

$$Q^+ = LD \cdot D + LD'Q,$$

乘法和加法逐位按逻辑运算解释。输出使能控制三态读出时，禁用的是对总线的驱动，而非寄存器内容。输出与门控制的禁用值为零，则与三态读出的高阻不同。

寄存器间经公共总线传送数据时，源寄存器输出使能决定总线来源，目的寄存器装载使能决定本沿是否接收数据。源使能和目的装载应保持到所需的建立、保持时间之后；读出使能不能代替写入时钟。

11.2 累加器与并行接口

累加器将寄存器内容 X 与输入 Y 相加，并把结果重新存入 X 。若先清零，每次累加使能 $AD = 1$ 时 $X^+ = X + Y$ （此处为算术加法）； $AD = 0$ 时保持。

另一方案允许直接装载第一操作数，优先级为装载、累加、保持：

$$X^+ = \begin{cases} Y, & LD = 1, \\ X + Y, & LD = 0, AD = 1, \\ X, & LD = AD = 0. \end{cases}$$

该方案用选择器决定寄存器输入，无须以清零作为每次运算的第一步。固定宽度累加须另行处理进位或溢出。并行输入输出接口用寄存器保存数据，使总线上短暂出现的数据在外部设备读出期间保持稳定。

11.3 移位寄存器及串并转换

移位寄存器在每个有效边沿把各位数据向规定方向移动一位。按 $Q_{n-1} \cdots Q_0$ 从左到右书写，右移为

$$Q^+ = \{R_{in}, Q_{n-1}, \dots, Q_1\},$$

左移为

$$Q^+ = \{Q_{n-2}, \dots, Q_0, L_{in}\}.$$

串行输入端分别补入右移时最高位、左移时最低位。右移通过串行输入装入完整数码时，应按最低位到最高位送入；左移则按最高位到最低位送入。

串入串出 (SISO) 可作延迟链，串入并出 (SIPO) 进行串到并转换，并入串出 (PISO) 进行并到串转换，并入并出 (PIPO) 进行整字传送。若在第一个沿装入第一级， n 位链的末级在第 n 个沿出现该位；相对第一级的输出时刻相差 $n - 1$ 个周期。讨论“延迟 n 周期”时须明确以输入准备时刻还是首次采样时刻为起点。

```

module shift4(
    input wire clk, clr,
    input wire [1:0] mode,
    input wire [3:0] d,
    input wire rin, lin,
    output reg [3:0] q
);
    always @(posedge clk) begin
        if (clr) q <= 4'b0000;
        else case (mode)
            2'b00: q <= q;
            2'b01: q <= {rin, q[3:1]};
            2'b10: q <= {q[2:0], lin};
            2'b11: q <= d;
            default: q <= q;
        endcase
    end
endmodule

```

上述模型为高有效同步清零，不能直接当作低有效异步清零的 74LS194。双向移位寄存器的并行装载、左移、右移需互斥，或由模式代码给出明确优先级。若写成独立使能的加权或形式，还须补上全禁用时的保持项。

11.4 环形与扭环形计数器

右移寄存器把最低位反馈到串行输入，可组成环形计数器。四位单一有效位循环为

$$1000 \rightarrow 0100 \rightarrow 0010 \rightarrow 0001 \rightarrow 1000.$$

在该循环中， n 位寄存器得到 n 个状态，各位直接给出依次有效的节拍。普通结构不能保证从任意状态进入单一有效位循环：0000、1111 为固定点，另有多有效位循环，故须预置或增加恢复逻辑。

把最低位的反反馈到右移串行输入构成扭环形计数器（Johnson counter）。四位有效循环为

$$0000 \rightarrow 1000 \rightarrow 1100 \rightarrow 1110 \rightarrow 1111 \rightarrow 0111 \rightarrow 0011 \rightarrow 0001 \rightarrow 0000.$$

它用 n 位形成 $2n$ 个有效状态，相邻有效状态仅改变一位。四位时八个节拍输出可取

$$\begin{array}{llll}
 Y_0 = Q'_3 Q'_0, & Y_1 = Q_3 Q'_2, & Y_2 = Q_2 Q'_1, & Y_3 = Q_1 Q'_0, \\
 Y_4 = Q_3 Q_0, & Y_5 = Q'_3 Q_2, & Y_6 = Q'_2 Q_1, & Y_7 = Q'_1 Q_0.
 \end{array}$$

两输入译码即可区分有效状态。但存在其他循环，通常也需要初始清零或自恢复。单比特状态变化减少译码过渡问题，不意味着任意连接、任意输入条件和复位瞬态都无冒险。

11.5 同步二进制计数器

计数器按输入脉冲循环经过预定状态，稳定循环的状态数称为模，记为 N 。用 n 个触发器最多有 2^n 个状态，但不代表每种计数器都使用全部状态。

同步计数器的全部触发器共用外部时钟。二进制递增加法计数中，最低位每次翻转；第 i 位只有当全部低位为 1 时翻转。以 Q_0 为最低位、 EN 为计数使能，可取

$$T_0 = EN, \quad T_i = EN \prod_{j=0}^{i-1} Q_j \quad (i \geq 1), \quad Q_i^+ = Q_i \oplus T_i.$$

用 D 实现时令 $D_i = Q_i^+$ ，用 JK 实现时令 $J_i = K_i = T_i$ 。

同步计数器不产生逐级时钟延迟的累积，但仍有时钟到输出延迟、组合逻辑延迟及建立时间。简化时钟周期条件为

$$T_{\text{clk}} \geq t_{CQ,\text{max}} + t_{\text{logic,max}} + t_{su}$$

并应另计时钟偏斜与裕量。最小路径也须满足保持约束。多位输出的传播时间不完全一致，其组合译码仍可能出现毛刺。

```
module counter4(
  input wire clk, clr, load, enp, ent, up,
  input wire [3:0] d,
  output reg [3:0] q,
  output wire rco
);
  always @(posedge clk) begin
    if (clr) q <= 4'd0;
    else if (load) q <= d;
    else if (enp && ent) begin
      if (up) q <= q + 4'd1;
      else q <= q - 4'd1;
    end
  end
  assign rco = ent && (up ? (q == 4'd15) : (q == 4'd0));
endmodule
```

该模型的控制优先级是同步清零、同步装载、计数、保持。递增终值为 15，递减终值为 0；rco 是当前终值指示，不是一个自动保持一个周期的寄存输出。

十进制递增计数使 0, 1, ..., 9 循环，计到 9 后下一沿回零，终值判断也改为 9。余 3 十进制计数使代码 3, 4, ..., 12 循环，复位装入 3，12 后回到 3，终值判断为 12。若允许装载非法代码，须说明是限制输入，还是规定非法状态恢复。

11.6 异步行波计数器与状态分析

异步计数器的外部时钟常只接最低位触发器，后续时钟由前一级输出提供，因此状态沿逐级传播。下降沿 T' 触发器级联、后一时钟接前一级 Q ，得到二进制递增值；改变边沿或连接 Q' 会改变计数方向，不能只凭“ T 触发器级联”判定方向。

三位递增行波从 011 到 100 时，真实过渡可能为

$$011 \rightarrow 010 \rightarrow 000 \rightarrow 100.$$

中间组合不是稳定计数状态，却可使外部译码产生错误短脉冲。累积传播延迟约随级数增加，各位稳定分频频率依次为 $f_{\text{clk}}/2, f_{\text{clk}}/4, \dots$ 。

分析异步电路时，应逐级判断是否收到有效边沿；仅满足 $J = K = 1$ 不代表本次外部时钟会让所有级翻转。表中次态要给出一次外部脉冲传播完成后的稳定结果，波形还须标出级间先后。

11.7 模 6 计数器的完整分析

例题：由 D 驱动方程判断功能及自启动

三位状态为 $Q_2Q_1Q_0$ ，共用上升沿时钟，驱动方程为

$$D_0 = Q'_0, \quad D_1 = Q_1Q'_0 + Q'_2Q'_1Q_0, \quad D_2 = Q_2Q'_0 + Q_1Q_0.$$

解 对每个现态代入，得到

现态	次态	现态	次态
000	001	100	101
001	010	101	000
010	011	110	111
011	100	111	100

有效循环为 $000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 101 \rightarrow 000$ ，故是模 6 加法计数器。未用状态 $110 \rightarrow 111 \rightarrow 100$ ， $111 \rightarrow 100$ ，能在有限脉冲后进入有效循环，因而在状态转移意义下可自启动。自启动不等于有确定上电初相位，确定相位仍须复位。

11.8 节拍电位与节拍脉冲

节拍发生器在每个周期内产生按次序有效的一组控制信号，可由模计数器加译码器、环形计数器或扭环形计数器构成。 N 个等长状态各占一个时钟周期时，每条单状态译码线的重复周期为 NT_{clk} 。

节拍电位持续一段控制窗口，例如指定源数据或操作方式；节拍脉冲提供具体执行边沿，例如装载寄存器。节拍电位应覆盖执行边沿及所需建立、保持时间。译码生成的脉冲含有传播延迟，不能按理想状态图直接当作可靠时钟。

课件自恢复环环节拍核心可写成

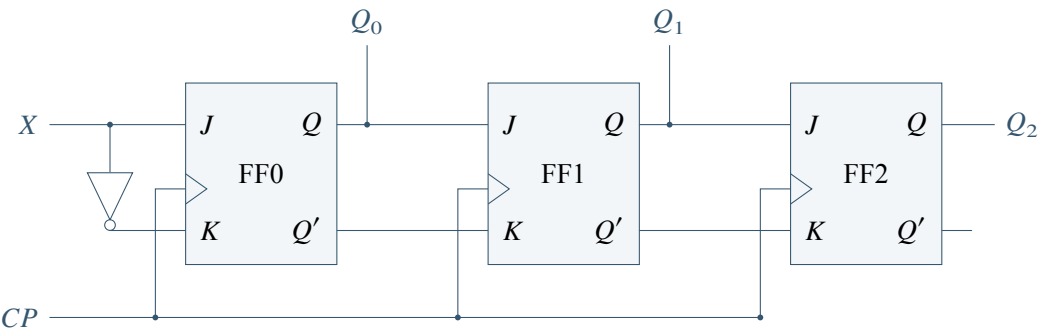
$$Q_0^+ = Q_1, \quad Q_1^+ = Q_2, \quad Q_2^+ = Q_3, \quad Q_3^+ = (Q_3 + Q_2 + Q_1)'.$$

0000 下一沿进入 1000，之后形成四个单一有效位的循环。对任意初态还应逐项检查； Q_3 的反馈不是普通环形的 Q_0 。用 SR 锁存器把某两个节拍的置位、复位事件转成较长的控制电位时，须保证两个控制输入不会同时有效，并按器件实际极性连接。

11.9 由 JK 接线识别同步移位寄存器

例题：三触发器串联电路的六拍状态

2024 秋期末回忆版，分析题 2。电路初态 $Q_0 = Q_1 = Q_2 = 0$ ，共用上升沿时钟。六个采样沿的输入 X 依次为 1,0,1,0,0,1。求逐拍输出并判断功能。



解 先读取接线，而非直接把三个触发器认作计数器：

$J_0 = X, K_0 = X'; \quad J_1 = Q_0, K_1 = Q'_0; \quad J_2 = Q_1, K_2 = Q'_1.$

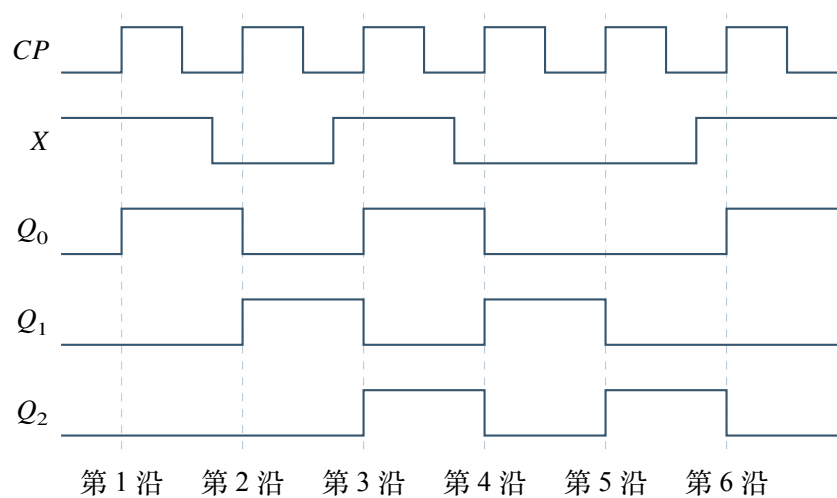
对一般接法 $J = U, K = U'$ ，代入特征方程得到 $Q^+ = UQ' + UQ = U$ 。所以三组 JK 在这里都实现 D 功能：

$Q_0^+ = X, \quad Q_1^+ = Q_0, \quad Q_2^+ = Q_1.$

右端全部取同一沿前的旧状态，不能让第二级读到本沿刚更新的 Q_0 。

脉冲	输入 X	沿前 $Q_2Q_1Q_0$	沿后 $Q_2Q_1Q_0$
1	1	000	001
2	0	001	010
3	1	010	101
4	0	101	010
5	0	010	100
6	1	100	001

输出波形如下，输入在采样沿之间改变，以避免建立、保持窗口：



该电路是三位同步串入并出移位寄存器，也可从 Q_2 串行读出。若 $X[k]$ 表示第 k 沿采样的输入，则 $Q_2[k] = X[k - 2]$ ($k \geq 3$)。首个输入在第 1 沿进入第一级，在第 3 沿到达第三级；从首沿到末沿相隔两个周期，不能与“经过三级存储”混为三周期。

12 同步状态机的分析与设计

12.1 有限状态机、Mealy 与 Moore 模型

有限状态机 (FSM) 由有限状态集合、输入集合、次态函数、输出函数及指定初态构成。电路状态总结了决定未来行为所需的历史信息。数字控制电路通常持续运行, 不要求具有终态或到终态停止。

Mealy 型为 $Q^+ = F(Q, X)$ 、 $Z = G(Q, X)$; **Moore** 型的次态仍可依赖输入, 输出为 $Z = G(Q)$ 。Mealy 输出标在边的“输入/输出”上, Moore 输出标在状态内。Mealy 输入变化可在同一周期改变输出; Moore 组合译码也可能在状态位交接时有毛刺, 需要寄存输出时应明确另加寄存器。

例题：带使能的两位计数状态机

次态为 $Q_0^+ = Q_0 \oplus EN$ 、 $Q_1^+ = Q_1 \oplus (Q_0 EN)$ 。当 $EN = 0$ 时保持, 当 $EN = 1$ 时 $00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00$ 。

若 $MAX = Q_1 Q_0 EN$, 输出与现态和输入有关, 为 Mealy 型; 若 $MAX_S = Q_1 Q_0$, 输出只由现态决定, 为 Moore 型。前者表示当前允许进位的终值条件, 后者表示已处于最大状态。

12.2 从需求建立原始状态表

方法：同步时序电路设计流程

1. 规定输入、输出、采样边沿、初态及输出的时间含义。
2. 根据必须记忆的历史建立原始状态图和状态表, 覆盖每个状态的全部输入。
3. 删除确定不可达状态, 合并等价状态, 保留功能及初态意义。
4. 分配状态编码, 选择触发器, 建立编码后的次态与输出表。
5. 由激励表求驱动函数, 化简并实现; 核对非法状态的恢复及输出。
6. 通过状态表、输入序列和边界条件验证, 再检查物理时序。

状态划分应仅保留需要的历史。例如串行二进制加法器每次输入 X_1, X_2 , 只需记住前一位的进位 c 。两个状态分别代表无进位、有进位, 输出和次态为

$$Z = X_1 \oplus X_2 \oplus c, \quad c^+ = X_1 X_2 + (X_1 \oplus X_2)c.$$

位流须从最低位开始, 初态 $c = 0$ 。同步模 5 可逆计数器则需五个计数状态; 输入 $X = 0$ 时递增、 $X = 1$ 时递减, 输出可定义为“状态 4 且递增, 或状态 0 且递减”的进借位条件。输出的含义与状态转移时刻必须一并说明。

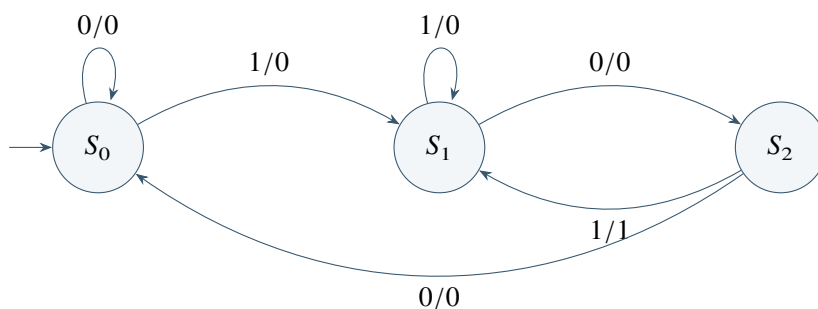
12.3 101 序列检测与重叠

序列检测每个有效时钟沿接收一个输入 X , 识别最近输入是否形成目标串。状态表示“已接收串的后缀中, 与目标前缀匹配的最长部分”。设三状态 S_0 (无非空匹配)、

S_1 (匹配 1)、 S_2 (匹配 10)。

现态	$X = 0$ 时次态/输出	$X = 1$ 时次态/输出
S_0	$S_0/0$	$S_1/0$
S_1	$S_2/0$	$S_1/0$
S_2	$S_0/0$	$S_1/1$

上表是可重叠 Mealy 检测器。完成 101 后，末尾的 1 可作为下一次匹配的起点，因此回到 S_1 。输入 10101 在第 3、第 5 位各输出一次。



不可重叠检测器在完成后丢弃本次已使用的三位，将 S_2 在 $X = 1$ 时的次态改为 S_0 ，仍输出 1。此时 10101 只在第 3 位报告，不能在第 5 位复用上一匹配的末位。

若采用 Moore 型，另设 S_3 表示已检测到 101， S_3 的输出为 1，其他状态为 0。可重叠时 S_3 收到 0 转到 S_2 ，收到 1 转到 S_1 ；不可重叠时，收到 0 转到 S_0 、收到 1 转到 S_1 。两种模型的输出时间须在波形中标明，不能把“进入检测状态”与“由当前边产生检测输出”混写。

12.4 等价状态与隐含表法

对于完全给定的确定状态机，两个状态等价，是指从它们出发，对任意后续输入序列产生相同输出序列。判别需同时满足：每个输入下当前输出相同；相应次态也等价。当前输出相同仅是必要条件。

隐含表法对全部不同状态对建立三角表：先把存在输出差异的状态对标为不等价；对其他状态对记录其各输入下的次态对。若记录中任何次态对已不等价，当前对也不等价。反复传播到不再变化，未被排除的状态对等价。只互相依赖且没有矛盾输出的封闭链可以保留。等价关系具有传递性，最后按最大等价类合并。

例题：课件七状态表的化简

表中的每项为“次态/输出”。

现态	$X = 0$	$X = 1$
a	c/0	b/1
b	f/0	a/1
c	d/0	g/0
d	d/1	e/0
e	c/0	e/1
f	d/0	g/0
g	c/1	d/0

解 输出向量先分为 $\{a, b, e\}$ (0,1)、 $\{c, f\}$ (0,0)、 $\{d, g\}$ (1,0)。c,f 的各次态完全相同，故等价。(a,b) 依赖 (c,f) 和自身，(a,e)、(b,e) 依赖同组内状态对，因此三者等价。

d,g 虽输出相同，但在 $X = 0$ 时分别转到 d,c，两者输出向量不同，故 d,g 不等价。令 $q_1 = \{a, b, e\}$, $q_2 = \{c, f\}$, $q_3 = \{d\}$, $q_4 = \{g\}$ ，得

现态	$X = 0$	$X = 1$
q_1	$q_2/0$	$q_1/1$
q_2	$q_3/0$	$q_4/0$
q_3	$q_3/1$	$q_1/0$
q_4	$q_2/1$	$q_3/0$

完全给定机的等价类互不重叠；不完全定义状态机的兼容关系需要其他处理，本讲义按课件主要范围讨论完全给定状态表。

12.5 余 3 码串行误码检测

例题：每四位判断一次的码制检测器

余 3 码最高位先输入，每接收第 4 位时判定该组是否有效。有效码为 0011 至 1100；非法码输出 $Z = 1$ ，有效码输出 $Z = 0$ ，随后重新接收下一组。

解 建立前缀树：初态 ε 代表已接收零位；每个输入形成下一层前缀。收到前三位时各有 2, 4, 8 个可能前缀，第四位只输出判断并返回初态，原始状态总数为 $1+2+4+8 = 15$ 。

最底层三个比特的的前缀，其第四位输出为：000 和 111 对 0,1 均输出 1；001 输出 1,0；010,011,100,101 均输出 0,0；110 输出 0,1。令它们的等价类为 A,B,C,D。前两位 01,10 均向 C,C 转移，亦可合并。保留各接收位数的阶段，得到十状态表：

状态意义	$X = 0$ 时次态/输出	$X = 1$ 时次态/输出
ε (零位)	$P_0/0$	$P_1/0$
P_0 (前缀 0)	$P_{00}/0$	$P_{01,10}/0$
P_1 (前缀 1)	$P_{01,10}/0$	$P_{11}/0$
P_{00}	$A/0$	$B/0$
$P_{01,10}$	$C/0$	$C/0$
P_{11}	$D/0$	$A/0$
A (000 或 111)	$\varepsilon/1$	$\varepsilon/1$
B (001)	$\varepsilon/1$	$\varepsilon/0$
C (010、011、100 或 101)	$\varepsilon/0$	$\varepsilon/0$
D (110)	$\varepsilon/0$	$\varepsilon/1$

前三位收到时输出一直为零，第四位输出才有判断意义。若接口还需区分“尚未判断”与“有效码”，可添加完成标志。帧边界必须由复位、有效使能或输入协议保证；不能把任意连续四位滑动窗口与分组检测混为一谈。

12.6 状态编码与触发器数量

N 个不同状态采用紧凑二进制编码时，最少位数为

$$k = \lceil \log_2 N \rceil, \quad 2^{k-1} < N \leq 2^k \quad (N > 1).$$

$N = 1$ 时可无状态存储位，也可按实现约定保留寄存位。使用更多状态位可能简化组合逻辑，如独热编码用每个状态一个位；最少触发器数量不必给出最小实现总代价。

课件的相邻编码规则是启发式方法：同一输入下次态相同的现态尽量相邻；同一现态在不同输入下的次态尽量相邻；输出相同的现态也可尽量相邻。这些目标可能冲突，通常优先考虑驱动函数化简，并让初态便于复位。编码必须互不相同，不能为满足相邻性把两个非等价状态编为同一码。

12.7 110 检测器：从状态化简到 JK 实现

例题：JK 触发器实现 110 序列检测

每个有效边沿接收一位 X 。识别到连续子串 110 时输出一 次 $Z = 1$ ，采用 Mealy 输出。

解 设 S_0 为无匹配， S_1 为末尾 1， S_2 为末尾 11。若另设完成状态 S_3 ，它下一输入的行为与 S_0 相同，可合并。最简状态表为

现态	$X = 0$	$X = 1$
S_0	$S_0/0$	$S_1/0$
S_1	$S_0/0$	$S_2/0$
S_2	$S_0/1$	$S_2/0$

分配 $S_0 = 00, S_1 = 10, S_2 = 11$ ，状态变量为 Q_2Q_1 。逐位按 JK 激励表填表，未用编码 01 先作化简无关状态：

X	Q_2	Q_1	Q_2^+	Q_1^+	J_2	K_2	J_1	K_1	Z
0	0	0	0	0	0	×	0	×	0
0	1	0	0	0	×	1	0	×	0
0	1	1	0	0	×	1	×	1	1
1	0	0	1	0	1	×	0	×	0
1	1	0	1	1	×	0	1	×	0
1	1	1	1	1	×	0	×	0	0

化简得到

$$J_2 = X, \quad K_2 = X', \quad J_1 = XQ_2, \quad K_1 = X', \quad Z = X'Q_1.$$

将实际驱动代回特征方程，检查所有状态：

$$Q_2^+ = XQ_2' + XQ_2 = X,$$

$$Q_1^+ = XQ_2Q_1' + XQ_1 = X(Q_2 + Q_1).$$

未用 01 在 $X = 0$ 时进入 00，在 $X = 1$ 时进入 11，一步即可恢复。然而在 01, $X = 0$ 时原输出 $Z = 1$ ，恢复期间可能给出一次无效检测。若要求非法状态不报检测，可把输出改为 $Z = X'Q_2Q_1$ ；合法状态上功能不变。

```

module detect110(
    input wire clk, rst_n, x,
    output wire z
);
    reg q2, q1;
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin q2 <= 1'b0; q1 <= 1'b0; end
        else begin
            q2 <= x;
            q1 <= x & (q2 | q1);
        end
    end
    assign z = (~x) & q2 & q1;
endmodule

```

此代码按化简次态用 D 触发器表达，与上述 JK 方案的状态行为相同。 z 为组合 Mealy 输出，在采样边沿前当 $x = 0$ 且现态为 11 时有效；边沿后状态回零。若需沿后保持一个完整周期的检测脉冲，应另用寄存器在该沿采样条件，并说明输出接口的时序。

12.8 非法状态、自启动与功能验证

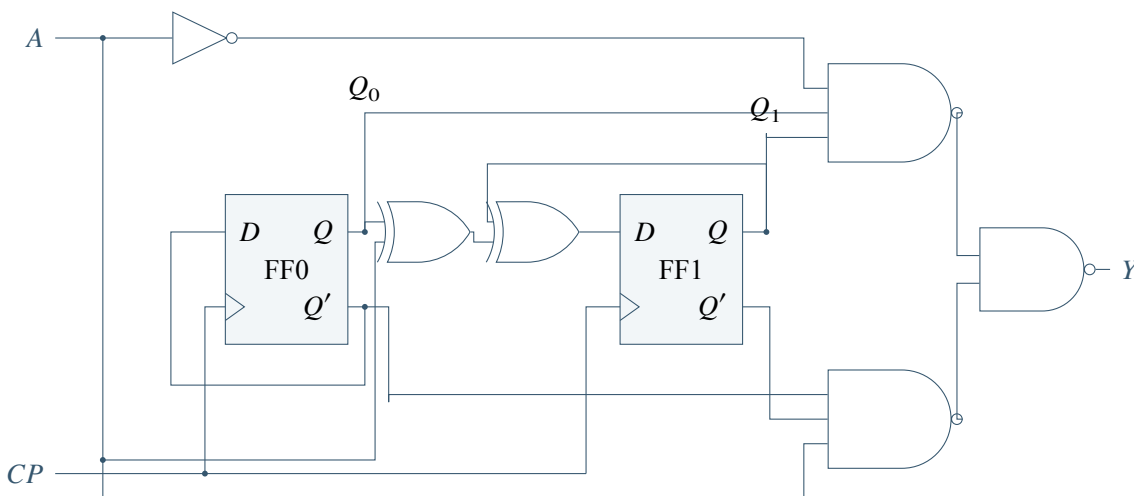
自启动是电路能从未用编码在有限更新后进入有效工作集合。它不是有确定的初态，也不能保证恢复途中输出正确。设计应分别检查：所有非法编码的实际次态；是否存在非法闭环或固定点；恢复输出是否允许；恢复是否依赖特定输入。

状态机验证可将编码实现的每个合法状态、每个输入组合，与抽象状态表逐项比较，再使用有代表性的连续输入验证历史意义。例如 110 检测输入 1110 只在最后一个 0 处报告，因为最后三位是 110；输入 110110 报告两次。非阻塞更新须用旧状态，避免在同一沿错误提前推进多个匹配阶段。

12.9 从两级 D 触发器电路识别可逆计数

例题：含异或门与与非门的同步电路

2024 秋期末回忆版，分析题 1。按下图求次态方程、输出方程、状态表及状态图。两触发器共用上升沿时钟， Q_1Q_0 为从高到低的现态， A 为控制输入。复位或起始状态须由外部条件确定。



图中实心点表示导线相接；没有实心点的交叉线互不连接。

解 第一级 D 端接自己的反相输出，所以每沿翻转。第二级 D 端来自两个异或门，依次加入 Q_0, A, Q_1 。因此

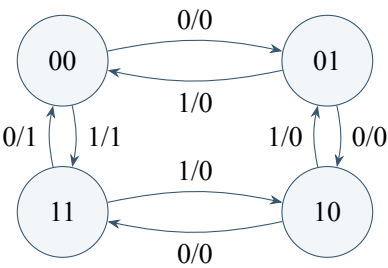
$$Q_0^+ = Q_0', \quad Q_1^+ = Q_1 \oplus Q_0 \oplus A.$$

输出部分的上、下与非门分别产生 $(A'Q_0Q_1)'$ 与 $(AQ_0'Q_1')'$ ；再经末级与非门，由德摩根律得到

$$Y = A'Q_0Q_1 + AQ_0'Q_1'.$$

逐项代入现态，不从输出方程推测次态：

现态 Q_1Q_0	$A = 0$: 次态/ Y	$A = 1$: 次态/ Y
00	01/0	11/1
01	10/0	00/0
10	11/0	01/0
11	00/1	10/0



边标记为 A/Y 。 $A = 0$ 时按 $00 \rightarrow 01 \rightarrow 10 \rightarrow 11 \rightarrow 00$ 递增； $A = 1$ 时反向递减。因此它是二位模 4 可逆计数器， Y 是当前方向下的终值标志：递增时现态 11 有效，递减时现态 00 有效。由于 Y 还依赖 A ，属于 Mealy 输出；稳定的终值电平不等于已经存储的一周期进位脉冲。

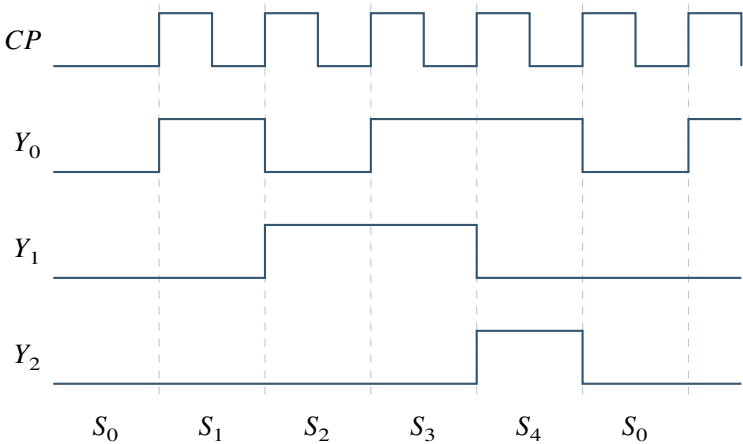
12.10 从多路输出波形设计 JK 状态机

例题：五拍循环的三路输出波形

依据 2024 秋期末回忆版设计题 2 的题型改编。规定在每个上升沿后，输出 $Y_2Y_1Y_0$ 循环为

000 $\rightarrow$ 001 $\rightarrow$ 010 $\rightarrow$ 011 $\rightarrow$ 101 $\rightarrow$ 000 $\rightarrow \cdots$.

复位后输出 000。用 JK 触发器实现，给出所需触发器数、状态分配、驱动方程及非法状态恢复方式。



解 先读有效沿之间的完整输出组合，再确定循环状态。五个输出组合各不相同，至

少需要五个状态；因 $2^2 < 5 \leq 2^3$ ，须用三个触发器。设内部状态为 $Q_2Q_1Q_0$ ，按自然升序分配：

状态	内部编码	次态编码	输出 $Y_2Y_1Y_0$
S_0	000	001	000
S_1	001	010	001
S_2	010	011	010
S_3	011	100	011
S_4	100	000	101

内部编码与输出不同，尤其 S_4 的编码 100 对应输出 101，不能把输出字直接当作状态分配。规定未用编码 101、110、111 在下一沿回到 000。根据现态与次态逐位查 JK 激励表：

现态	次态	J_2	K_2	J_1	K_1	J_0	K_0
000	001	0	×	0	×	1	×
001	010	0	×	1	×	×	1
010	011	0	×	×	0	1	×
011	100	1	×	×	1	×	1
100	000	×	1	0	×	0	×

再把三个非法编码的清零转移纳入驱动要求，用激励表中的无关输入化简，可得

$$\begin{aligned} J_0 &= Q_2', & K_0 &= 1, \\ J_1 &= Q_2'Q_0, & K_1 &= Q_2 + Q_0, \\ J_2 &= Q_1Q_0, & K_2 &= 1. \end{aligned}$$

代回特征方程检查全部八个编码：

$$\begin{aligned} Q_0^+ &= Q_2'Q_0', \\ Q_1^+ &= Q_2'(Q_1'Q_0 + Q_1Q_0'), \\ Q_2^+ &= Q_2'Q_1Q_0. \end{aligned}$$

在 $Q_2 = 1$ 的三个非法编码中，上式全部为零，确实一步恢复 000。有效状态输出可取 $Y_0 = Q_0 + Q_2, Y_1 = Q_1, Y_2 = Q_2$ ；若还要求恢复期间输出 000，令有效编码标志 $V_s = Q_2' + Q_1'Q_0'$ ，再给三个输出分别与上 V_s 。最后逐拍核对五个合法状态的输出，就得到题设波形。

回忆版原图没有标明重复周期边界，个别起始跳变也未与时钟边沿对齐。本例重新明确周期、有效边沿与初态，演示同类题的完整设计方法，不把这些补充条件当作原试题已给条件。

13 中规模芯片与序列信号发生器

13.1 计数器功能表与控制优先级

中规模芯片内部包含计数与控制逻辑，使用时以外部功能表为依据。课件常用器件如下，计数输入均须满足具体器件的有效边沿和时序要求。

型号	正常计数模	装载方式	清零方式
74LS160	10	同步、低有效	异步、低有效
74LS161	16	同步、低有效	异步、低有效
74LS162	10	同步、低有效	同步、低有效
74LS163	16	同步、低有效	同步、低有效
74LS192	10	异步、低有效	异步、高有效
74LS193	16	异步、低有效	异步、高有效
74LS190	10	异步、低有效	无独立清零端
74LS191	16	异步、低有效	无独立清零端

74LS160–163 的计数为同步上升沿，计数使能 ENP, ENT 必须都为 1；装载不依赖两者同时启用。清零的优先级高于装载，装载高于计数，其他情况保持。同步控制需要控制电平与有效边沿共同满足；异步清零在有效电平到来后立即生效，无须等待时钟。

四位二进制型号的进位标志为 $RCO = ENT \cdot Q_D Q_C Q_B Q_A$ ；十进制型号在正常 BCD 循环中于 9 给出对应终值标志。输出顺序 $Q_D Q_C Q_B Q_A$ 中 Q_D 最高、 Q_A 最低。控制规律参考课件，并用 TI 74LS160A–163A 资料核验。

13.2 反馈清零与反馈置数

方法：终值判断的时机

若稳定循环要求 $0, 1, \dots, M - 1$ ，异步清零方案通常译码 M ，达到后立即清零；同步清零或同步装载方案须译码 $M - 1$ ，使下一沿回零。两者稳定循环都含 M 个状态，异步方案另经过极短的复位触发过渡。

以模 10 为例：74LS161 可在出现 1010 时异步清零；从初态零开始计数时，可用 $Q_D Q_B$ 的首次成立判断 10，清零引脚接 $(Q_D Q_B)'$ 。这是在正常可达计数路径上使用部分译码；若必须从任意非法初态恢复，应检查其他状态，或使用完整 $Q_D Q'_C Q_B Q'_A$ 译码。

74LS163 同步清零须在 1001（9）期间使清零有效，使下一沿直接回 0000。完整终值译码为 $Q_D Q'_C Q'_B Q_A$ ，同步清零引脚接其反；若在 1010 才触发，下个边沿才清零，将多保留一个状态，模值成为 11。

同步置数归零法令装载数据为 0000， $LD_n = (Q_D Q'_C Q'_B Q_A)'$ ，可用 74LS161 或 163 构成模 10。同步装载的触发状态同样是 9，不能因为 161 具有异步清零就把它

装载也当作异步。

预置起点 P 、终点 T 并在 T 的下一沿装回 P ，循环状态数为 $T - P + 1$ 。例如 16 进制芯片从 6 数到 15 后同步装回 6，得到模 10 循环；输出代码为 6 至 15，并不是 BCD 的 0 至 9。

13.3 计数器级联：模 256 与模 60

计数器级联必须明确用前一级终值控制使能，还是作为后一级时钟。共用外部时钟的级联利用低位终值条件驱动高位计数使能；行波方式使时钟沿逐级产生，须考虑极性和累积延迟。

例题：两片 74LS161 构成模 256

两片分别存低四位 Q_L 与高四位 Q_H 。两片共用上升沿时钟，低片允许每沿递增，高片只在低片沿前为 15 时递增。

解 低片 $ENP_L = ENT_L = 1$ ，高片 $ENP_H = 1, ENT_H = RCO_L$ 。两片装载端保持非有效，清零端按同一复位要求控制。则

$$Q_L^+ = (Q_L + 1) \bmod 16, \quad Q_H^+ = (Q_H + [Q_L = 15]) \bmod 16.$$

方括号表示条件成立为 1、否则为 0。共 $16 \times 16 = 256$ 个稳定状态。两片以上可使用逐组进位条件，使能链延迟仍须满足建立时间，不能认为共用时钟就没有组合路径限制。

两片 74LS160 构成模 60 时，个位正常十进制计数，十位只在个位为 9 时递增；检测“十位为 5 且个位为 9”，在下一沿同时同步装载零。十位使能来自个位终值，装载优先级高于使能。因此稳定序列为 00, 01, ..., 59, 00。若检测 60 才触发同步装载，则会把 60 作为额外稳定状态。

13.4 双时钟与方向控制的可逆计数器

74LS192/193 分别是十进制、二进制的双时钟加减计数器。清零是高有效，低有效装载可异步预置；正常时，加计数要求减时钟保持高并在加时钟上升沿更新，减计数要求加时钟保持高并在减时钟上升沿更新。无须动作的那一路不能固定接低。

其进位和借位输出与计数时钟相位有关，可接后级相应计数时钟完成扩展。清零优先及双时钟条件依据 TI 74LS193 说明核对。课件把该清零端写作 CLRN，但功能表为高有效；本讲义用 CLR 表示其高有效清零，避免仅按名称误判。

74LS190/191 用单一时钟和方向端 D/U ， $D/U = 0$ 递增、 $D/U = 1$ 递减，计数使能低有效；没有独立清零端，可异步装载零。它们提供终值指示及行波时钟输出，不能套用 192/193 的独立进、借位端。方向和使能的切换须遵循器件时序条件，详见 TI 74LS191 说明。

例题：比较器控制可调模值

用 74LS85 与 74LS193 构成模 N 递增计数器， $1 \leq N < 16$ 。

解 比较器一侧输入设为 N ，另一侧接计数输出 Q ，独立比较的级联输入设置为相等。

正常从零开始，达到 $Q = N$ 时，相等输出使 193 的高有效异步清零端有效，于是立即回零。稳定循环为 0 至 $N - 1$ ； N 是清零触发过渡状态。上电和改变 N 后可能有 $Q > N$ ，单独相等反馈不一定立即恢复，需复位或为新需求另设恢复条件。

13.5 74LS90 的二、五、十进制连接

74LS90 内部有模 2 段和模 5 段，分别由 CP_A, CP_B 的下降沿触发。 Q_A 为模 2 段输出， $Q_DQ_CQ_B$ 为模 5 段输出。两清零控制同时为高时可复位到零；两置 9 控制同时为高时可置为 1001，同时控制的优先级须按功能表使用，常规计数时保持置位、清零输入非有效。

8421 BCD 模 10 连接：外部时钟接 CP_A ， Q_A 接 CP_B ，按 $Q_DQ_CQ_BQ_A$ 读出，形成 0000 至 1001 的稳定循环。5421 BCD 模 10 连接：外部时钟先接 CP_B ，模 5 终值转回零时的 Q_D 下降沿驱动 CP_A ，按 $Q_AQ_DQ_CQ_B$ 读出，稳定循环依次为

0000, 0001, 0010, 0011, 0100,
1000, 1001, 1010, 1011, 1100.

例题：两片 74LS90 实现模 45 的两种思路

一种构成十进制两位计数，在到 45 时共同异步清零；另一种先模 5 再模 9，使模数乘积为 45。

解 第一种使两片各为 8421 十进制，低片在 $9 \rightarrow 0$ 时的 Q_D 下降沿驱动高片时钟。完整检测高位 4、低位 5，把检测信号接两片的清零控制，稳定计数 00 至 44，45 短暂出现后复位。

第二种低片仅用模 5 段，高片作模 9；低片每 5 次输入产生一次终值回转边沿，使高片更新，高片达到 9 时异步复位。 $5 \times 9 = 45$ ，但其状态代码并非普通两位十进制的 00 至 44。两种方案都需初始复位、正确边沿和足够的复位脉宽。

同理，模 7 计数器可用异步反馈在 7 时清零，并将三个计数位译码成七个节拍。异步反馈触发状态及译码毛刺应与稳定模值分别说明。74LS90 结构及连接参见 TI 74LS90 资料。

13.6 74LS194 双向移位寄存器及级联

74LS194 的输出顺序记为 $Q_AQ_BQ_CQ_D$ ，模式端为 S_1S_0 ，低有效 CLR_n 异步清零。清零非有效时，按上升沿更新：

S_1	S_0	工作方式	下一状态 $Q_AQ_BQ_CQ_D$
0	0	保持	$Q_AQ_BQ_CQ_D$
0	1	右移	$D_{SR}Q_AQ_BQ_C$
1	0	左移	$Q_BQ_CQ_DD_{SL}$
1	1	并行装载	$ABCD$

这里左右按书写位置定义，而 Q_A 并非字母 A 所天然决定的最高权；它是本表左侧位置。芯片连线与模式依据 TI 74LS194A 数据手册核对。课件概念框图偶尔标有额外 LD，本器件并行装载由 $S_1S_0 = 11$ 决定，不另加独立装载引脚。

构造八位双向移位时，两片共用时钟、清零和模式。把两片组成一个从左到右的八位字：右移时右片串入端接左片最右位，左片串入端接外部右移输入；左移时左片串入端接右片最左位，右片串入端接外部左移输入。两个方向的边界连接都须完成。

右移模式把 Q'_D 接 D_{SR} 可构成四位扭环形计数器；初始清零进入八状态循环。普通环形则反馈 Q_D ，须并行预置一个有效位。

13.7 七位串并转换器的哨兵控制

课件用两个移位寄存器组成八位向量 $F_7 \cdots F_0$ ，用 F_7 控制装载或左移。异步清零后 $F_7 = 0$ ，选择并行装载；下一沿装入

$$F_7 \cdots F_0 = 1111110D_6.$$

接着 $F_7 = 1$ ，改为左移，依次从串行端输入 $D_5, D_4, \dots, D_0$ 。各沿后的正确内容依次为

$$\begin{aligned} &111110D_6D_5, \quad 11110D_6D_5D_4, \\ &1110D_6D_5D_4D_3, \quad 110D_6D_5D_4D_3D_2, \\ &10D_6D_5D_4D_3D_2D_1, \quad 0D_6D_5D_4D_3D_2D_1D_0. \end{aligned}$$

初始装入的零是哨兵位，移到 F_7 后指示七位数据已全部接收，此时低七位是并行输出。再下一沿装载下一组；因此接收协议须在每组第一沿提供 D_6 ，之后提供剩下六位，并在完成窗口读取结果。课件中间一行少移动了一位，以上按逐沿移位关系修正。

13.8 计数器加选择器或 ROM 产生序列

序列信号发生器循环输出固定的位串，长度为一个规定周期的位数。若该位串本身由更短位串重复而成，最小周期应先约简；计数器方案可直接以最小周期设置模数。

长度为 L 的位串 $s_0s_1 \cdots s_{L-1}$ ，用模 L 计数器按 $k = 0, \dots, L-1$ 循环，输出 $Y = s_k$ 。可由 MUX 输入 $I_k = s_k$ 实现，也可由 ROM 地址 k 读出对应位；多路输出序列用 ROM 各数据位分别存放。

例题：110100 序列的三种实现

输出从复位后的第一状态起，循环为 1, 1, 0, 1, 0, 0。

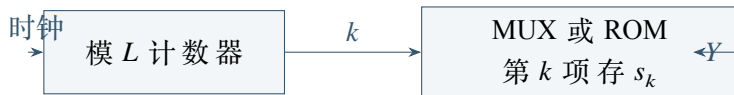
解 计数器加 MUX：用 74LS163 作同步模 6，状态 5 时使下一沿回零；八选一的 $I_0, I_1, I_2, I_3, I_4, I_5$ 接 1, 1, 0, 1, 0, 0。 I_6, I_7 接确定值，同时规定计数器非法状态恢复，不能让数据端悬空。

计数器加 ROM：三位地址，八个一位表项，在地址 0 至 5 存 1, 1, 0, 1, 0, 0，其余表项按恢复期间输出要求设置。ROM 法便于多路输出和较长序列，但存储规模随地址位数增长。

用 D 触发器直接实现：以 $Q_2Q_1Q_0 = 000, 001, 010, 011, 100, 101$ 编码六个状态，采用前章模 6 的驱动方程，输出

$$Y = Q_2'Q_1' + Q_1Q_0.$$

它在六个合法状态上依次为 1, 1, 0, 1, 0, 0, 110, 111 按实际次态进入合法循环。复位决定序列起点，不能只证明循环相同就忽略相位。



13.9 移位寄存器加反馈产生序列

用 n 位移位寄存器产生长度 L 的周期串时， $2^n \geq L$ 只是必要条件。把循环位串连续 n 位作为状态窗口；若同一窗口的后续位不同，就不能仅由该窗口决定反馈，须增大 n 或另加状态信息。

例题：00010111 序列的反馈设计

采用三位左移寄存器，状态 $Q_2Q_1Q_0$ 的最高位 Q_2 为输出。

解 循环窗口为

$$000 \rightarrow 001 \rightarrow 010 \rightarrow 101 \rightarrow 011 \rightarrow 111 \rightarrow 110 \rightarrow 100 \rightarrow 000.$$

八个状态两两不同，按 $(Q_2, Q_1, Q_0)^+ = (Q_1, Q_0, L_{in})$ ，各状态的下一补入位为

状态	L_{in}	状态	L_{in}
000	1	011	1
001	0	111	0
010	1	110	0
101	1	100	0

以 $Q_2Q_1Q_0$ 为编号，反馈成一项为 0, 2, 3, 5，可化简为

$$L_{in} = Q_2Q_1'Q_0 + Q_2'Q_1 + Q_2'Q_0' = \sum m(0, 2, 3, 5).$$

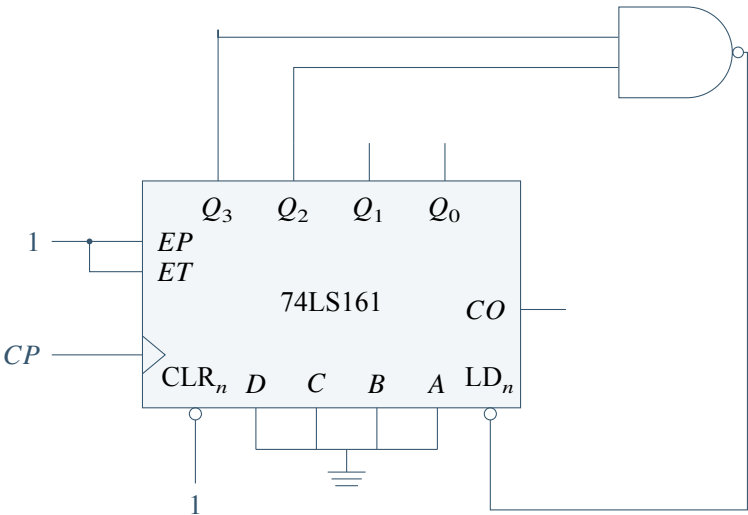
因此可用逻辑门实现，也可用八选一将 0、2、3、5 号数据端接 1，其余接 0；用 74LS138 则对应低有效译码输出经与非门合成。最高位输出依次为 00010111。八个三位状态全部在同一循环中，无未用二值状态；任意二值初态对应不同相位，确定相位仍需初始化。

若同一状态窗口重复且后续位相同，可考虑合并，但仍须检查规定周期及相位含义。环形、扭环形适于规则序列；计数器加 MUX/ROM 适于按索引输出；反馈移位实现依赖窗口能唯一决定下一位。选择方法须同时考虑状态数、资源和所需输出。

13.10 从反馈装载接线判断稳定模值

例题：74LS161 反馈装载计数器

2024 秋期末回忆版，简答题 5。 $Q_dQ_cQ_bQ_a = Q_3Q_2Q_1Q_0$ ，初态 0000。计数使能 $EP = ET = 1$ ，并行输入 $DCBA = 0000$ ，异步清零在计数期间不生效；与非门把 Q_3, Q_2 译码到低有效装载端。求稳定状态序列与计数模值。



解 装载控制的引脚电平是 $LD_n = (Q_3Q_2)'$ 。 $Q_3Q_2 = 11$ 时该端为零；其余情况为一。74LS161 的并行装载是同步的，须在下一个上升沿读取低有效装载条件。

从 0000 递增，第一次达到 $Q_3Q_2 = 11$ 的状态是 1100，即十进制 12。进入 12 的那一沿之前，现态为 11（二进制 1011），装载端仍为一，因此这次正常计数到 12。到达 12 后装载端变为零，下一沿才装入 0000：

沿前状态	LD_n	沿后状态	本沿动作
1010 (10)	1	1011 (11)	正常加一
1011 (11)	1	1100 (12)	正常加一
1100 (12)	0	0000 (0)	同步装载零

所以稳定序列为 $0 \rightarrow 1 \rightarrow \cdots \rightarrow 12 \rightarrow 0$ ，共有 13 个状态，是模 13 计数器。1100 维持一个完整的稳定计数区间，不能按“达到 12 就立即清零”误判为模 12。其终值 CO 只有在 1111 且相应使能条件满足时有效；本循环未达到 1111，不能直接把 CO 当作模 13 终值信号。

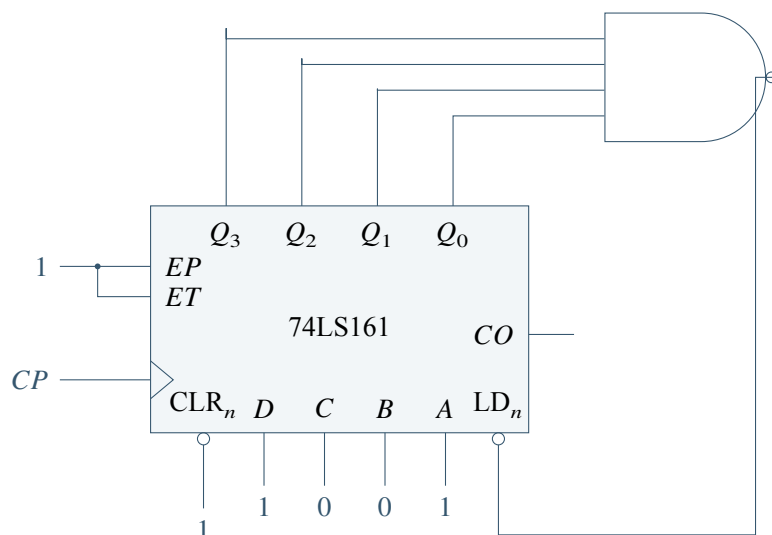
例题：设计二进制 1001 至 1111 循环计数器

2024 秋期末回忆版，简答题 7。用 74LS161 使稳定计数范围为 1001_2 至 1111_2 ，下一沿由 1111 回到 1001。说明并行输入、反馈条件、初态设置与模值。

解 并行输入须接 $DCBA = 1001$ 。计数允许且清零不生效时，设置

$$EP = ET = 1, \quad LD_n = (Q_3 Q_2 Q_1 Q_0)' = CO'.$$

最后一项相等依赖 $ET = 1$ ，不能在任意使能条件下照用。电路重绘如下，四输入与非门在 1111 产生装载低电平：



启动时先用外部装载控制使 $LD_n = 0$ ，在一个上升沿装入 1001，然后交给终值反馈；单独异步清零只会装入 0000，不能获得要求的初态。正常循环为

$$1001 \rightarrow 1010 \rightarrow 1011 \rightarrow 1100 \rightarrow 1101 \rightarrow 1110 \rightarrow 1111 \rightarrow 1001,$$

共有 $15 - 9 + 1 = 7$ 个稳定状态，模值为 7。进入 1111 后还须等待下一有效沿，期间它是有效终值状态。若从 0000 直接启动，会先经历额外的 0 至 8，不能把这段启动过程说成题设范围内的循环。

14 实验资料、代码阅读与验证约定

14.1 资料来源与版本对应

本实验部分依据HOA 课程页面及其链接资料整理。课程页面的路径不是实验年份的证明。资源下载区的labs/2020包含六次旧版实验；labs/2024/exp3/README.md包含 2024 年数码管实验说明。页面“相关实验仓库”链接到capoo-fan/class-experiment，其中指导书明确出现 2025 年的检查日期，且具有六次实验的设计、约束和仿真代码。本部分以这套 2025 年资料为代码讲解主线，历史版本单列补充。以下要求描述所读取的资料，不表示 2026 年课程已采用相同题目。

实验	2025 年链接仓库	labs/2020及 2024 年补充
1	3-8 译码器；四位加减多路复用器	拨码开关控制 LED；3-8 译码器
2	D 触发器；八个八位寄存器	低有效复位、低有效写使能；结构化寄存器文件
3	四档频率、双向、按键启停流水灯	1 秒流水灯；四位格雷码计数器
4	未消抖/消抖按键计数；0 至 30 循环	倒计时、开关值、班级、学号；2024 版为 0 至 20 饱和计数
5	UART 发送hitsz+ 学号	Moore/Mealy 检测01011
6	UART 收发、最近六字符显示、字符串匹配选做	记忆游戏

注意：开发板和个人信息必须与资料版本对应

2025 年资料的板载 100MHz 时钟接Y18；2020 年 EGO1 资料接P17。复位极性、数码管极性、拨码开关编号和约束文件不能跨板照搬。参考代码中的学号和尾号属于原作者；实际提交时必须替换成使用者本人的信息。尚未给出个人学号，本讲义不编造该信息。

14.2 代码文件与硬件结构

配套“实验代码/原始参考”按六次实验保留下载到的.v和.xdc文件，未经功能改写；“整理示例”放置本讲义的规范化教学示例。原代码和改写示例通过目录及正文说明区分，完整设计代码可在配套文件中查阅。各实验的top或testbench名称可能重复，应分别建立工程，不能把全部实验一次加入同一设计。

设计源文件描述可综合的电路；仿真源文件产生激励并检查输出；约束文件指定引脚、电气标准和时钟。always ##5 clk = ~clk;产生仿真时钟；它不生成 FPGA 内部的晶振。##10、\$display、\$finish用于测试，不能当作上板电路中的延时方案。实际延时依靠时钟和计数器。

读代码时先确定端口及有效电平，再寻找always @(posedge clk ...)中的状态，

随后读取组合逻辑和模块实例，最后追踪状态更新条件与输出。在一个时钟边沿上，非阻塞赋值`<=`先计算右侧旧值，再统一更新左侧；同一边沿工作的多个模块因此可以按寄存器连接关系分析。

```
// A two-stage pipeline: both right-hand sides use old values.  
always @(posedge clk) begin  
    stage1 <= data_in;  
    stage2 <= stage1;  
end
```

若边沿前`stage1=0`、`data_in=1`，边沿后`stage1=1`而`stage2=0`。用`=`依次改写两句时，第二句会读取刚改写的`stage1`，不能按两级流水线理解。组合过程通常用`=`，时钟过程通常用`<=`。`reg`表示过程变量，完整组合赋值的`reg`仍可以综合成组合电路。

14.3 验证层次与仿真证据

功能仿真核对逻辑与时序协议；静态检查核对未声明信号、位宽、锁存器、多驱动及赋值习惯；综合和实现核对资源、时钟、引脚和时序；上板核对实际电平、抖动及连线。功能仿真通过只证明测试激励覆盖到的行为，不能替代后两层。

测试程序宜在`negedge clk`改变输入，在`posedge clk`之后留出非阻塞赋值更新时间再检查。检查使用`!==`可以把`x`或`z`也作为失败，而普通`!=`可能产生未知的条件值。

```
@(negedge clk); en = 1'b1; d = 8'hA5;  
@(posedge clk); #1;  
if (q !== 8'hA5) $fatal(1, "write failed");
```

波形分析应说明“哪个边沿、哪些输入、预期输出、实际输出及依据”。至少覆盖复位、正常路径、暂停或失能、边界和关键失败路径。缩小计数参数可加快仿真，但必须注明缩小比例，上板参数仍按实际 100MHz 时钟计算。

15 Lab 1: 组合逻辑与加减多路复用器

15.1 要求与文件对应

2025 年实验包括 3-8 译码器和多路复用器。链接仓库的设计文件mux.v实现四位加减选择，testbench.v给出十组激励，pin.xdc连接开关和 LED。译码器在指导书中给出接口与功能表，仓库未提供相应的独立设计文件；后文给出完整教学实现。

多路复用器输入为en、mux_sel及两个四位无符号数input_a、input_b。失能en=0时输出4'hF；使能且mux_sel=0时相加；使能且mux_sel=1时相减。四位输出只保留结果低四位。

```
module mux(
    input wire en, mux_sel,
    input wire [3:0] input_a, input_b,
    output reg [3:0] output_c
);
    always @(*) begin
        if (!en) output_c = 4'hF;
        else if (!mux_sel) output_c = input_a + input_b;
        else output_c = input_a - input_b;
    end
endmodule
```

always @(*)表示依赖信号变化后重新计算，三条分支覆盖全部二值控制组合，因此不需要存储旧输出，也不推断锁存器。它不是“先加后减”的执行流程，而是综合成由选择控制的组合数据通路。

15.2 位宽、溢出与测试输入

例题：四位运算的实际结果

input_a=4'hF、input_b=4'h1时，加法输出4'h0。input_a=4'h1、input_b=4'h2时，减法输出4'hF。后一结果表示模 16 的低四位，不是带负号显示的十进制数。

若需保留加法进位，必须扩展输入与输出，例如wire [4:0] total = {1'b0,a} + {1'b0,b}；。实验接口规定四位输出时，丢弃进位符合该接口，不应擅自把端口改成五位。

原仿真声明十位switch，再按以下连接拆分输入。

```
reg [9:0] switch;
assign {en, mux_sel, input_a, input_b} = switch;
```

拼接左侧总宽度为 $1 + 1 + 4 + 4 = 10$ ，switch[9]对应en、switch[8]对应mux_sel，其后两组各四位。10'b1_0_1000_0100表示使能、加法、8、4，结果应为 12。下划线只用于可读性，不占位宽。

原十组激励覆盖使能和加减，但没有检查进位截断、负差及全部输入，也没有自动判错。完整穷举包含 $2 \times 2 \times 16 \times 16 = 1024$ 组，逐组按真值规则计算预期四位结果；配套验证文件完成此检查。

15.3 3–8 译码器与有效电平

指导书规定 `en=3'b100` 时使能，地址 `data_in` 选中的输出位为 0，其余为 1；失能时输出全 1。这相当于低有效输出译码器，而不能把“选中”一律写成 1。

```
module decoder_38(  
    input wire [2:0] en, data_in,  
    output wire [7:0] data_out  
);  
    wire active = en[2] & ~en[1] & ~en[0];  
    assign data_out = active ? ~(8'b00000001 << data_in)  
                           : 8'hFF;  
endmodule
```

例如地址 5 时，`8'b1 << 5` 为 `8'h20`，取反后 `data_out=8'hDF`。八位常量确保移位与取反都发生在确定宽度内。2020 年指导书的某些示例片段省略使能或输出反相，并把连续赋值输出声明为 `reg`；完整传统 Verilog 实现应使用 `wire` 并遵从实际功能表。译码器穷举 $8 \times 8 = 64$ 组地址及使能组合。

15.4 约束、检查与提交内容

上板验证先检查所有顶层端口都与本板引脚对应，再确认 `IOSTANDARD`。总线端口在 Tcl 中使用花括号保护，例 `[get_ports {input_a[0]}]`；括号中的端口名须与 RTL 完全相同。组合模块不需要伪造内部时钟。

报告需包含输入输出波形与解释、RTL 分析图、综合后电路图。两类图对应不同抽象阶段；综合后结构取决于优化与目标器件，不能仅凭源代码形式断言一定出现某种门或 ROM。原约束只可用于相同开发板，换板时需重新核对每个引脚。

16 Lab 2: 触发器与寄存器文件

16.1 D 触发器：复位、写入与保持

2025 年dff/dff.v使用高有效异步复位clr和高有效同步使能en，输出为q。复位无需等待时钟；写入必须在上升沿满足使能；失能时保持。

```
always @(posedge clk or posedge clr) begin
  if (clr) q <= 1'b0;
  else if (en) q <= d;
end
```

没有else表示时钟触发器保持原值。原文件末尾的q <= q;功能等价但可以省略。d在两个时钟边沿之间变化不会立即改变q；复位在边沿之间变为1 却会立即清零。异步复位释放在实际硬件中还应满足恢复、移除时序，功能仿真不显示亚稳态。

16.2 八个八位寄存器与地址选择

reg8file/reg8file.v中reg [7:0] regfile [7:0];声明八个元素，每个元素八位，共 64 位数据存储。前面的[7:0]是每个元素的位宽，后面的[7:0]是数组索引范围。wsel与rsel均为三位，分别选择写地址、读地址。

```
reg [7:0] regfile [0:7];
integer i;
always @(posedge clk or posedge clr) begin
  if (clr) begin
    for (i=0; i<8; i=i+1) regfile[i] <= 8'h00;
  end else if (en) regfile[wsel] <= d;
end
assign q = regfile[rsel]; // q is declared as output wire
```

该规范化片段把原always @(*) q <= regfile[rsel];改成连续赋值：读端是组合逻辑，不应额外采用非阻塞更新。异步清零全部元素通常要求触发器资源，不能直接断言这种写法推断出块 RAM。

写地址决定哪一个寄存器在时钟边沿接收数据，其余七个保持。读地址改变时，读输出经过组合延迟跟随对应元素，不需要等下一个边沿。读写同址时，边沿前读出旧内容，非阻塞更新后读出新内容；这是本实现的行为，不代表所有 RAM 端口都如此。

操作	en	地址与数据	边沿后的结果
复位	任意	clr=1	八个元素均为 0
写入	1	wsel=3,d=A5	第 3 项为A5
保持	0	wsel=3,d=5A	第 3 项仍为A5
读取	任意	rsel=3	输出第 3 项；读不修改存储

16.3 2020 年结构化要求的独立实现

旧版规定`clrn=0`异步清零, `wen=0`同步写入; 其中`wen`名字未带`_n`, 但功能表明明确低有效。旧版还明确要求实例化译码器、D 触发器及多路选择器, 不能仅用数组行为描述替代该结构要求。

```
module dffe(input wire clk, clrn, wen, d,
            output reg q);
    always @(posedge clk or negedge clrn)
        if (!clrn) q <= 1'b0;
        else if (!wen) q <= d;
endmodule

module reg8(input wire clk, clrn, wen,
            input wire [7:0] d, output wire [7:0] q);
    genvar b;
    generate for (b=0; b<8; b=b+1) begin: bits
        dffe u_ff(.clk(clk), .clrn(clrn), .wen(wen),
                  .d(d[b]), .q(q[b]));
    end endgenerate
endmodule
```

再实例化八个`reg8`。第 j 项的低有效写使能为`we_n[j] = wen | (wsel != j)`, 只有总写使能有效且地址匹配时为0。读多路选择器把`rsel`指定的八位输出送到`q`。配套`core_examples.v`给出完整连接。`generate`是在`elaboration`阶段展开硬件, 八次循环不是运行八拍。

旧指导书建议不约束`rsel[2]`以应对开关不足。实际上不能让顶层输入悬空并期待确定值; 可以在板级包装模块把该位固定为`1'b0`, 明确上板只可读取低四个地址, 而核心仿真仍验证全部八个地址。

16.4 验证与易混概念

至少检查边沿之间异步复位、失能保持、八个地址分别写读、覆盖同址写入及复位后全部清零。原仿真只主要访问地址1和2, 不能证明全部地址独立。静态`for`复位八项可以展开综合; 测试中等待时间、检查循环属于仿真过程。

寄存器文件的写使能只控制写入, 不控制读端。`en=0`时仍可读取。两套复位和使能的极性必须在端口表和代码中一致, 不能只替换变量名。

17 Lab 3: 可调频率与方向的流水灯

17.1 要求与计数参数

2025 年实验要求: 100MHz 时钟, $\text{rst}=1$ 异步复位后仅 $\text{led}[0]$ 亮; 按 button 在启动、停止间切换, 复位后等待启动; freq_set 选择四档移位频率; $\text{dir_set}=0$ 右移、 $\text{dir_set}=1$ 左移。指导书要求三级寄存器边沿检测, 本实验不要求消抖。

令输入频率 $f_c = 100 \text{ MHz}$ 、每次移位间隔 T , 一轮计数长度 $N = f_c T$ 。从 0 数到 $N - 1$ 共有 N 个时钟周期, 因此终值为 $N - 1$ 。

freq_set	频率	间隔	周期数 N	终值
00	1000Hz	1ms	100000	99999
01	100Hz	10ms	1000000	999999
10	20Hz	50ms	5000000	4999999
11	5Hz	200ms	20000000	19999999

原文件参数 cnt_500hz 实际数值是 1_000_000-1 , 对应 100Hz, 应改名为 cnt_100hz 以免误读。最大值小于 2^{25} , 25 位足够; 原 26 位也正确, 只是多留一位。

17.2 三级采样与启停标志

```

reg sig_r0, sig_r1, sig_r2;
wire pos_edge = sig_r1 & ~sig_r2;
always @(posedge clk or posedge rst) begin
    if (rst) begin
        sig_r0 <= 0; sig_r1 <= 0; sig_r2 <= 0;
    end else begin
        sig_r0 <= button;
        sig_r1 <= sig_r0;
        sig_r2 <= sig_r1;
    end
end
always @(posedge clk or posedge rst) begin
    if (rst) flag <= 1'b0;
    else if (pos_edge) flag <= ~flag;
end

```

输入连续为 1 并传播到第二级而尚未传播到第三级时, pos_edge 为 1; 下一拍第三级也为 1, 脉冲结束。flag 每次上升沿脉冲翻转一次, 持续按住不会反复翻转。前两级采样有助于同步异步输入, 仍不能消除机械抖动; 多个真实上升沿仍可能翻转多次。

原代码没有显式声明 pos_edge , 传统 Verilog 会隐式建立一位 wire。这种隐式声明掩盖拼写错误, 整理示例显式声明并使用 ``default_nettype none` 检查未声明连线。

17.3 终值脉冲和循环移位

```

wire cnt_end = flag && (cnt >= cnt_max);
always @(posedge clk or posedge rst) begin
    if (rst) cnt <= 0;
    else if (cnt_end) cnt <= 0;
    else if (flag) cnt <= cnt + 1'b1;
end
always @(posedge clk or posedge rst) begin
    if (rst) led <= 8'h01;
    else if (cnt_end) begin
        if (!dir_set) led <= {led[0], led[7:1]};
        else led <= {led[6:0], led[7]};
    end
end
end

```

cnt_end是单拍时钟使能，不是 50% 占空比的新时钟。两段过程都由同一个clk触发，计数归零与移位使用同一拍边沿前的条件。右循环移位把低位移到最高位，8'h01变成8'h80；普通led >> 1会移入 0 并丢掉唯一的 1，不能替代循环移位。

停止时cnt和led保持。再次启动时从剩余计数继续，因此“恢复后第一次移位”不必再等待完整 T 。切换到较短间隔且现值已超过新终值时，>=使其下一拍归零移位；使用==可能错过终值。是否切换频率时重置相位是接口约定，原实现选择保留现计数并用大于等于处理。

17.4 频率含义、仿真与旧版要求

5Hz 表示每秒移位 5 次，八个位置轮一遍需 $8/5 = 1.6$ 秒；单个 LED 重新亮起的周期为八次移位间隔。旧版“其余灯亮过以后再亮”的描述不能误读成相邻两次点亮同一个 LED 只隔七个总周期。

原仿真把四档周期数缩小 1000 倍，并用 10ns 的时钟：第一档每 100 拍移位一次，间隔 1 微秒，第二档 10 微秒，第三档 50 微秒，第四档 200 微秒。报告应测量相邻移位的时间差，并明确这不是上板 1ms 至 200ms 的真实间隔。

2020 版要求独立divider产生 1Hz 时钟，再驱动flowLED，其结构约束与本版单时钟使能方案不同。若采用翻转法得到 1Hz、50% 占空比，则每 50 000 000 拍翻转一次，终值为49_999_999；若每 100 000 000 拍翻转一次，实际为 0.5Hz。生成的新时钟在工程中还需相应时钟约束和正确时钟资源，不能仅看 RTL 波形。

18 Lab 4: 数码管扫描、消抖与十进制计数

18.1 显示布局和模块划分

2025 版从左到右为DK7至DK0：最高两位显示学号末两位；其后两位统计未消抖的S3上升沿；中间两位统计消抖后的S3上升沿；最低两位以 0.1 秒间隔从 0 数到 30，再回 0。S2每按一次在暂停与继续间切换；SW0=0关闭显示但内部逻辑继续运行；S1=1异步复位所有时序逻辑。

设计由top.v、debounce.v、edge_detect.v、led_ctrl_unit.v组成。top.v分成输入处理、三组计数和显示拼接。led_ctrl_unit读取 32 位display，八个四位字段分别对应八个位置，最后形成led_en和led_cx。

```
assign display = {dig7,dig6,dig5,dig4,dig3,dig2,dig1,dig0};
assign led_en = SW0 ? anode_out : 8'hFF;
assign led_cx = SW0 ? segment_out : 8'hFF;
```

位号dig0位于拼接最右侧、总线最低四位。SW0只在输出处门控，因而关闭后重新打开可看到计数继续后的新值。这是“关闭显示”，不是“暂停计数”。原学号尾号为 48，使用前需替换。

18.2 动态扫描及段码约定

refresh_cnt每 100000 拍归零并令三位anode_select加一；每个位置保持 1ms，一轮八位为 8ms，每位刷新 125Hz。三位索引自然从 7 回 0。

```
if (refresh_cnt == time_max) begin
    refresh_cnt <= 0;
    anode_select <= anode_select + 1'b1;
end else refresh_cnt <= refresh_cnt + 1'b1;
```

以anode_select=2为例，选择display[11:8]，位选为8'b11111011，仅第 2 位为 0。本版位选低有效，段选按[7:0]={a,b,c,d,e,f,g,dp}排列，也为低有效。字符 0 点亮 a 至 f、熄灭 g 及 dp，所以段码为8'h03；字符 7 点亮 a、b、c，码为8'h1F。

数字	0	1	2	3	4	5	6	7	8	9
段码	03	9F	25	0D	99	49	41	1F	01	09

原字符 0 的旁注8'b1100_0011与实际8'h03不一致，应以逐段推导的码值为准。此模块只定义 0 至 9，其他输入默认全灭，适用于本实验十进制显示；要显示 A 至 F，必须补充译码。2020 版 EGO1 段选及板级位选为高有效，并有两组独立段总线，不可照搬本版低有效八位选模板。

理想 RTL 中同时切换段选、位选不显示路径延迟；上板若有重影，可在换位时先关闭全部位选、更新段码，再启用下一位。消隐拍属于显示时序改进，不改变各位置

所对应的数字。

18.3 连续稳定计数消抖与单次事件

原消抖模块保存state作为最近观察到的候选电平，输入不同时更新候选并清计数；连续相同足够久后将候选写入button_out。100MHz 下cnt_max=2_000_000约为20ms。原计数从0增长到cnt_max后下一拍才更新输出，准确时刻还包含候选捕获拍；不能宣称恰好2000000拍。

原button_in直接进入判断逻辑，没有先经过同步寄存器。规范化示例先二级同步，再对同步后的电平作连续稳定计数。

```
if (sync2 == stable) count <= 0;
else if (count == STABLE_CYCLES-1) begin
    stable <= sync2;
    count <= 0;
end else count <= count + 1'b1;
```

这里STABLE_CYCLES为接受一次与当前稳定状态不同的新电平所需的连续周期数。任何一次回到旧稳定值都会清零，短抖动不会通过；释放也必须稳定后才接受。

消抖输出是电平，持续按住期间为1；计数器必须使用该电平的上升沿事件。edge_detect延迟一拍并比较sig_r0 & !sig_r1，得到单拍脉冲。若直接if (button_out) count <= count+1;，持续按住会每个时钟加一。

18.4 BCD 进位与 0 至 30 循环

BCD 计数用独立四位数码保存个位和十位，个位9之后置0并使十位加1。原自动计数在十位3、个位0时归零，因而序列是00,01,...,29,30,00，共有31个值；一轮为3.1秒。

```
if (tick_100ms && !pause) begin
    if (tens == 4'd3 && ones == 4'd0) begin
        tens <= 0; ones <= 0;
    end else if (ones == 4'd9) begin
        ones <= 0; tens <= tens + 1'b1;
    end else ones <= ones + 1'b1;
end
```

原两组按键计数只对个位做9回0，十位一直四位加一，到10以后产生非BCD十位，显示默认熄灭。若约定只显示末两位，须在tens==9 && ones==9时两位同时回0，形成模100计数。讲义的完整示例明确采用该约定。

原pause仅阻止数字更新，0.1秒节拍计数继续运行，因此恢复后第一次加一可能不足0.1秒；Lab 3的暂停连同分频计数一起保持，两者的暂停语义不同。报告需解释自身实现，不能只写“计数暂停”。

18.5 验证范围与版本差异

仿真至少覆盖八个扫描位置及段码、短毛刺拒绝、两次稳定按下、持续按住一次计数、稳定释放、09 → 10进位、29 → 30 → 00、暂停和显示关闭时内部继续计数。SW0=0时应观察位选全失能而不是display清零。

2024 补充版的布局为“学号尾号、开关中 1 的个数、消抖按键计数、0 至 20 计数”，不是本版的未消抖计数。其 0 至 20 计数到 20 保持，按S2重启；重启应同时清数值和 0.1 秒子计数器，避免首个间隔不足 0.1 秒。人口计数用逐位累加得到 0 至 8，4 位输出足够，累加中间结果须采用足够位宽。2020 版则显示倒计时、开关十六进制值、班级和学号，并要求两个指定小数点常亮。

19 Lab 5: UART 发送与字符串调度

19.1 实验要求与帧格式

2025 版要求 9600 波特率、三段式状态机，不断发送hitsz+ 个人学号。两个字符串发送之间至少等待 0.2 秒，同一字符串中相邻字符发送间隔不超过 0.01 秒。S1为高有效异步复位。仓库uart_send.v实现单字节发送，top.v调度 15 个 ASCII 字符并等待；该 15 字符长度来自原作者十位学号，实际学号长度变化时需同步改末地址与指针位宽。

UART 使用独立于系统时钟的串行位时序。此处采用8N1：空闲 1、起始位 0、8 个数据位、无校验、停止位 1，数据最低位先发。一个字节占 10 个位周期。

```
// Frame order in time: start, data[0],...,data[7], stop.
wire [9:0] frame = {1'b1, data, 1'b0};
// frame[0] is transmitted first.
```

发送 ASCII 字符'A'时数据为8'h41，实际八个数据位按时间为1,0,0,0,0,0,1,0。发送十六进制数8'h0A不等于发送两个字符'0'、'A'；后者须发送8'h30与8'h41两个字节。

19.2 波特率计数与状态转移

100MHz 系统时钟下，每位周期数取

$$N_b = \left\lfloor \frac{100\,000\,000}{9600} \right\rfloor = 10416, \quad f_b = \frac{100\,000\,000}{10416} \approx 9600.614 \text{ bit/s.}$$

相对 9600 的误差约 0.0064%。从 0 数到 10415，因此原cnt_max=10415正确。注释“10417”与该实现不符，应统一为 10416 拍。

状态	输出或动作	离开条件
IDLE	输出 1；接受valid并锁存data	请求有效后进入START
START	输出 0，起始位	一个baud_tick后进入DATA
DATA	输出data_buf[bit_cnt]	每个位末位号加一；位 7 结束进入STOP
STOP	输出 1，停止位	一个位周期后回IDLE

状态寄存器用时钟过程更新current_state；组合过程根据current_state、valid、baud_tick和bit_cnt产生next_state；输出过程产生dout。三段式描述强调职责分离，输出过程是否寄存还需明确，不能凭块数直接判定 Moore 或 Mealy。

```
if (current_state == IDLE && next_state == START)
    data_buf <= data;
```


这条锁存使发送期间外部data改变不影响当前帧。原dout根据旧现态在边沿上寄存，所以相对于current_state改变延后一拍；数据位号也在同一边沿更新，输出读取旧位号。逐拍跟踪时必须包括这拍延迟，不能只读状态图便宣称输出在状态跳转的同一拍已改变。

19.3 字符串 ROM 与顶层调度

top.v以pointer访问string_rom, case为每个地址给出 ASCII。它是组合查表，不是按时钟逐行执行。三个调度状态为send_char、wait_char和s_delay。在发送状态产生单拍uart_valid，等待一个字符帧时间后推进指针，最后一个字符后进入 0.2 秒等待。

一个字符帧约为 $10 \times 10416 / 100\,000\,000 = 1.0416$ 毫秒，15 字符约 15.624 毫秒；再等待 0.2 秒，字符串轮次约为 0.215624 秒，另有调度拍开销。char_wait_max=10*10416 用于估算忙碌时长，但固定等待必须与发射器的请求采样、输出延迟严格一致。

方法：用完成反馈推进发送

更清晰的接口为valid、ready、busy、done：只在valid && ready的边沿接受字节；发送完成时给出单拍done；字符串调度器收到done再推进pointer。这样位周期改变时无需在顶层重新估算十个位的计时。课程指定接口不能改时，可增加内部完成信号或保留经验证的等待逻辑，并明确接口限制。

原接口没有ready/busy。valid在非IDLE状态出现时会被忽略；持续拉高则可能在回到空闲时重复发送。因此一次请求应只高一拍，并由上层保证当前可以接受。

19.4 测试程序的边界与报告内容

原测试用例包括A5、3C、FF、00、5A，逐位比较起始、数据及停止位。原测试按以下表达式计算等待参数：

```
localparam DIVIDER = CLOCK_FREQ / BAUD_RATE - 1;
```

RTL 每位实际 10416 拍，测试每次却等待 10415 拍；短帧可能仍通过，但比较时刻会逐位提前。规范测试应以实际位长计时，在每位中心采样。

除原用例外，应验证输入数据在帧内改变、忙碌时请求、相邻帧边界、发送期间复位及字符串间隔。报告给出完整一帧、现态/次态、位计数、波特率计数、锁存数据和串行输出，而不能只展示FF连续高电平。

20 Lab 6: UART 接收、显示与字符串匹配

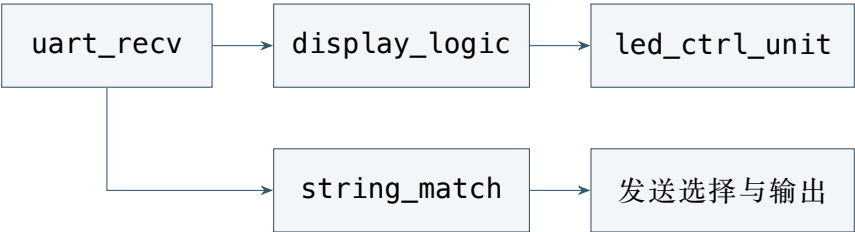
20.1 必做要求与顶层连接

2025 版必做内容为 9600 波特率收发：接收电脑端 ASCII 字符并在数码管显示最近六个字符，不足六个时高位全灭；剩余两位显示接收字符累计数的末两位；按S3一次发送一次个人学号，按键需消抖；S1为异步复位。指导书规定顶层只做实例化和模块连接，复杂功能放入子模块。

原top.v实例化下表所列模块。

模块	功能
debounce, edge_detect	按键稳定过滤与单次事件
uart_recv	串行帧接收，输出数据与有效事件
display_logic	ASCII 译码、最近六字符和累计数
led_ctrl_unit	八位扫描及空白字符显示
send_ctrl	按键请求后的字符串发送调度
string_match	前缀状态机与回复字符请求
mux	两条发送位流的模式选择

接收数据一路进入显示、另一路进入匹配；两套发送输出经SW0选择送往uart_tx。



原send_ctrl发送的是hitsz+ 原作者学号，而本实验必做要求为“个人学号”。Lab 5 的前缀不能直接当成 Lab 6 的要求；交付前应移除前缀或明确验证方接受的格式，并替换个人数据。

20.2 接收状态机与位中心采样

接收引脚是异步输入，应先二级同步。原uart_recv.v直接用din判断和采样，功能仿真可能正常，硬件仍有亚稳态传播风险；同步不会消除亚稳态本身，但可以降低其传播概率。

接收四态为IDLE、START、RECV、STOP。空闲检测到 0 进入起始态；等待半位并再次检查 0，以拒绝短低电平毛刺；然后每隔一整位采一个数据位；最后在停止位中点检查 1，输出数据和单拍valid。

相对起始下降沿时刻	采样内容	位位置
$0.5T_b$	起始位复核	应为 0
$1.5T_b$	第一个数据位	data[0]
$2.5T_b$ 至 $8.5T_b$	后续七位	data[1]至data[7]
$9.5T_b$	停止位	应为 1

表中 $T_b = 10416$ 个系统周期；实际实现增加同步和检测延迟，通常仅若干系统拍，远小于一个串行位。原START && cyc_half清零计数，故接收状态每次cyc_flag发生在下一数据位的中点，而不是数据边界；需要以起始沿为基准累加时间理解。

原接收器到停止位时间就输出valid，没有确认din==1，无法拒绝坏停止位。规范化uart_clean.v先同步，并验证停止位，坏帧只复位接收过程、不拉高有效信号。只对帧有效的数据更新显示和计数。

20.3 ASCII 译码、六字符移位与空白编码

display_logic.v用case(recv_data)把'0'至'9'和'A'至'F'转换成十六进制数码，同时兼容小写a至f；其他字符令valid_char=0。处理条件为valid && valid_char，因此原代码仅统计可显示字符。指导书只测试0至F，但累计“接收到的字符个数”的范围仍需约定；本讲义采用所有帧有效字节计数、仅可显示字符进入六槽显示，以便区分物理接收与显示过滤。

普通十六进制值占四位，却没有额外码表示空白。原代码每槽用五位，5'h1F表示全灭；0_0000至0_1111表示0至F。八槽拼接形成40位display_data，不同于Lab 4的32位。

```
if (valid && valid_char) begin
    slots[5] <= slots[4]; slots[4] <= slots[3];
    slots[3] <= slots[2]; slots[2] <= slots[1];
    slots[1] <= slots[0]; slots[0] <= {1'b0, hex_out};
end
```

非阻塞赋值保证每个槽取得旧的右邻字符，新字符进入slots[0]。复位时六槽全部为空白。连续接收1,2,A后，高位仍为空白，低端显示12A；不能把空白初始化为数字0。

原recv_count为八位，从255加一回0。即使显示时取recv_count / 10再模10，以及recv_count模10，第256个字符仍会显示00而非56。要只显示累计计数的末两位，应直接模100计数或用两位BCD，明确99 -> 00，这样任意长度输入都保留正确末两位。

20.4 字符串匹配：前缀状态与输入事件

选做一要求检测start、stop、hitsz并分别回复ASCII'1'、'2'、'3'；未匹配回复'0'；必须用状态机且可连续测试。仓库选择这项，未提供选做二矩阵键盘计算器的

设计代码。本讲义只整理已有实现及其正确性，不编造不存在的计算器代码。

string_match.v有 13 个状态：初始态，s,st,sta,star,start,sto,stop,h,hi,hit,hits,hitsz。状态表示已读字符后缀与目标前缀的匹配长度；valid=0时保持，不应在每个 100MHz 时钟把同一个字符重复读入。

```
if (valid) begin
  case (current_state)
    star_state: if (recv_data == "t" || recv_data == "T")
      next_state = start_state;
    sto_state:  if (recv_data == "p" || recv_data == "P")
      next_state = stop_state;
    hits_state: if (recv_data == "z" || recv_data == "Z")
      next_state = hitsz_state;
    // Full code supplies fallback transitions for every input.
  endcase
end
```

此片段只展示成功边，不是可独立使用的完整组合过程。原完整代码还有s/S、h/H重起前缀等失败转移；每个输入都必须给出次态。失败不能一律归初始，例如读到hits...等含可继续前缀的输入时须保存实际后缀。

匹配输出属于 Mealy 事件：current_state=star_state且当前有效字符为t时，就产生回复'1'，不必等进入start_state再读一个新字符。flag记录本行已有成功匹配，行终结字符到来时如未匹配才回复'0'。

20.5 终结符、忙碌反馈与模式切换

任意长度输入在尚未终结前不能判定“未匹配”。原代码以CR=8'h0D或LF=8'h0A为终结。若电脑发送CRLF，CR清除flag后紧跟LF，可能为已成功的行额外产生一次'0'请求；若发射器恰好忙碌，这次请求又可能被丢弃。两种缺陷会掩盖彼此，不能据串口只看到一个字节判断正确。

明确的策略是以LF结束一行而忽略CR，或把CRLF合并成一个终结事件。若仍选择单CR终结，测试软件必须采用该约定，并测试连续行、空行和一行多个命中。

原匹配器的回复请求直接送入没有busy反馈的uart_send。在上一回复发送中产生的请求会丢失。规范化设计将回复字符入队，只在发送器ready时出队；有限队列须定义满时行为。更基本的替代是按实验约定限制输入节奏，并在讲义和测试中明确该限制，不能暗称可处理任意速率连续命中。

顶层SW0直接多路选择两个独立发送器的位流，若在一帧中途切换可能截断帧。合理方案是在空闲时锁存模式，并由一个公共发送器服务两个请求源；若保留原代码，模式只应在两发送器空闲时改变。

20.6 完整验证路径

接收先通过原十组字节用例，再测试短起始毛刺、坏停止位、背靠背帧和帧中复位。显示测试空白、1 至 6 字符、第 7 字符挤出旧字符、大小写、非法字符及 255/256 字符计数边界。发送测试短抖动、长按只发一次、第二次按键、忙碌请求与学号内容。匹配测试三个关键词、大小写、噪声前缀、连续输入、未匹配行、CRLF 及发送拥塞。

每个测试应同时检查数据和有效事件，既防止“数值正确但重复接收”，也防止“有脉冲但数据错误”。报告的结构图应标出每个模块的时钟、复位、有效信号与数据方向；顶层连接正确不等于各子模块的接口时序已匹配。

21 历史 Lab 补充：格雷计数、序列检测与记忆游戏

21.1 格雷码计数器

2020 年 Lab 3 第二题要求四位格雷码循环计数，仅仿真，观察内部二进制计数和格雷输出并覆盖复位、使能。设四位二进制状态为 `binary`，格雷码为 `binary ^ (binary >> 1)`。

```
always @(posedge clk_i or negedge rst_n_i)
  if (!rst_n_i) binary <= 4'h0;
  else if (en_i) binary <= binary + 1'b1;
  assign gray_o = binary ^ (binary >> 1);
```

二进制 3 → 4 对应 0011 → 0100，多个二进制位变化；格雷码为 0010 → 0110，仅一位变化。15 → 0 对应 1000 → 0000，循环边也只变一位。逐项测试相邻码异或是否为独热值，同时检查 `en_i=0` 保持。

格雷码并不消除亚稳态。组合转换 `binary ^ (binary >> 1)` 可能因二进制触发器延迟不同而产生毛刺；跨时钟域使用时常把格雷结果寄存后同步，并满足相应路径约束。课程中的理想码表性质与真实物理波形需区分。

21.2 01011 序列：状态意义及转移表

2020 年 Lab 5 要求分别设计 Moore 及 Mealy 检测器，检查八位字内是否含 01011；`set_i` 开始新检测，结果命中后保持到下一次开始。需锁存输入字、明确从最高位至最低位输入状态机，并仅消耗八位，不把前一字后缀接到下一字前缀。

令 `S0` 表示无匹配前缀，`S1, S2, S3, S4` 分别表示已匹配 0, 01, 010, 0101。Mealy 转移表如下，`hit` 仅在读最后一个 1 时有效：

现态	输入 0 次态	输入 1 次态	命中条件
S0	S1	S0	无
S1	S1	S2	无
S2	S3	S0	无
S3	S1	S4	无
S4	S3	S0	输入 1

`S4` 遇 0 得到 01010，其中最长的可继续的目标前缀后缀为 010，故回 `S3`，不能一律回 `S1`。01011 末尾没有非空目标前缀后缀，命中后回 `S0`。Moore 增加 `S5` 表示完整命中，其输出为 1；`S5` 遇 0 去 `S1`、遇 1 去 `S0`。

Mealy 的瞬时命中为 `(state==S4) && bit_valid && bit_in`；Moore 命中为 `state==S5`。输出若寄存，须说明对应的采样边沿。字级持久结果可用 `detect_o <= detect_o | hit`；新 `start` 优先清零；Moore 在最后一位命中时须保留一拍完成态或根据最终次态捕获，不能在处理位 7 的边沿立即强制回初态而丢失命中。

验证全部 256 个字并与软件规则“八位字符串含01011”比较，其中原指定11011111为不命中，01010111命中。串行附加题每个消抖set_i事件接收一位，命中计数只在接受事件上加一；显示一位数时必须明确超过 9 后的处理。

21.3 记忆游戏：需求拆解与数据格式

2020 年 Lab 6 要求复位显示八个 0；S0启动生成五个互不相同的五位八进制数并存储，以 1 秒间隔播放；S1进入按地址查看；S2进入匹配输入；S3每次确认采样SW[2:0]；S4复位。五位八进制数用 15 位表示，每一位八进制数占 3 位。数码管十六进制译码仍可显示 0 至 7。

生成的五个数彼此不同，不要求一个数内部五个数码都不同。示例44444因此合法。可采用自由运行计数器采样或非零种子的 LFSR 产生候选，候选写入前与已经保存的 0 至 $k-1$ 项逐项比较，重复则重新生成；不能只用不同地址便宣称数据互异。上电固定种子的固定取样时序可能每次给出相同五项，应按题意让启动时刻影响取样，并记录不属于密码学随机源。

控制流程可设WAIT_START -> GENERATE -> PLAY -> WAIT_VIEW -> VIEW -> MATCH。PLAY保存播放索引和 1 秒计数，每满 1 秒推进一项；第 5 项显示满 1 秒后保持并等待S1。输入模块保存最近五个八进制数码，每次确认事件更新窗口：

```
wire [14:0] candidate_window = {window[11:0], sw[2:0]};
if (confirm_pulse) begin
    window <= candidate_window;
    // Compare candidate_window, not the old window.
end
```

若在同一时钟过程更新window后用window比较，非阻塞语义会比较旧窗口，结果晚一位。还需记录已经输入的数量，未满五位之前不得以初始化的前导零误报匹配。

21.4 旧指导书中的地址与闪烁歧义

五项按示例分别编号 0 至 4，输入03读取第四项17531；因此有效地址应为0 <= address < 5。指导书却写“地址大于 5 时显示 F”，没有定义地址 5，并与五项存储的范围不一致。本讲义按五项、零起始编号统一采用address >= 5为非法，显示八个 F；若当年教师另有地址约定应按明确补充修改。

显示“地址-匹配数”可统一为八个字符，例如4--44444，两条短横线占两位。题目示例只写一条短横，剩余位未指定；本讲义以八位布局明确处理，不能将此排版选择当作唯一官方格式。

匹配失败显示八个以 4Hz 闪烁的 0。若 4Hz 指完整亮灭周期，则半周期 0.125 秒，在 100MHz 下每 12500000 拍翻转亮灭，终值12_499_999；若每 0.25 秒翻转则完整闪烁为 2Hz。报告应明确采用的定义。每次S3均可继续更新窗口和匹配，成功后也不能锁死而阻止下一次输入。

21.5 模块接口与验证安排

建议划分按键同步消抖、周期节拍、候选生成及去重、五项存储、游戏控制、五位滑动匹配、显示编码和扫描。所有模块统一采用同一系统时钟，低速行为以时钟使能驱动。跨模块事件用单拍脉冲，数据在事件有效的边沿被接收。

验证应检查五项互异、数值 15 位宽和种子规则、五次播放顺序及时间、所有有效/非法地址、恰满五位时首次匹配、连续滑窗成功与失败、闪烁完整周期，以及每个状态中复位都回等待态。存储写入后立即读取时需注意实现的读延迟，不用仿真无限等待掩盖接口未定义。

本历史综合题没有在所读取 2020 资料中提供完整 RTL 答案；上述是依据指导书整理的设计与关键代码，不冒称原作者已完成的工程。2025 综合题的原始代码则在配套目录完整保留。

22 主要结论与术语

22.1 核心公式与适用条件

内容	结论及条件
按位计数	$N = \sum d_i r^i$; 数符范围与基数一致
二进制到格雷	最高位不变, 其余 $g_i = b_{i+1} \oplus b_i$
标准表达式	$F = \sum m(\mathcal{O}) = \prod M(\mathcal{Z})$; 固定变量次序, 成一与成零编号互补
德摩根	$(A + B)' = A'B'$, $(AB)' = A' + B'$; 反号作用范围明确
香农展开	$F = XF_1 + X'F_0 = (X' + F_1)(X + F_0)$
卡诺圈	2^k 格合法圈消去 k 个变量; 圈 1 读与或, 圈 0 读或与
静态冒险	指定两级结构下, 相邻目标格需由共同项覆盖; 单输入变化判据
全加器	$S_i = a_i \oplus b_i \oplus C_{i-1}$; $C_i = a_i b_i + (a_i \oplus b_i)C_{i-1}$
全减器	$D_i = a_i \oplus b_i \oplus B_{i-1}$; $B_i = a'_i b_i + a'_i B_{i-1} + b_i B_{i-1}$
MUX	$Y = \sum m_k(\mathbf{S})I_k$; 编号由选择端位序决定
ROM 与 LUT	n 输入、 b 输出 ROM 需 $2^n b$ 位; k 输入一位 LUT 有 2^k 表项
奇偶校验	偶校验位为数据归约异或, 奇校验位为其反; 检出奇数位翻转
触发器	SR: $Q^+ = S + R'Q$, $SR = 0$; D: $Q^+ = D$; JK: $Q^+ = JQ' + K'Q$; T: $Q^+ = T \oplus Q$
时钟使能	$Q^+ = CE \cdot D + CE'Q$; 禁用时保持
计数器	N 是稳定循环的状态数; n 触发器最多 2^n 状态
反馈截断	从 0 计数: 异步清零译码 M , 同步归零通常译码 $M - 1$
二进制状态编码	$k \geq \lceil \log_2 N \rceil$; 最少状态位不等于最少总逻辑
Mealy 与 Moore	Mealy 输出依赖现态和输入, Moore 输出只依赖现态

22.2 常用符号与术语

符号或术语	本讲义的含义
A', Q'	逐位逻辑取反; 不是次态
Q^+	次态; 课件也记 Q^{n+1} 或 Q^*
$AB, A + B$	与、或; HDL 分别用 &、

符号或术语	本讲义的含义
$\oplus, \odot$	异或、同或
m_i, M_i	编号 i 的最小项、最大项, $M_i = m'_i$
$d_i, \times$	不完全给定函数的无关项, 具体符号结合上下文识别
MSB、LSB	最高、最低有效位; 明确位宽和位序
C_{i-1}, C_i	全加器的输入、输出进位; 外部最低输入为 C_{-1}
B_{i-1}, B_i	全减器的输入、输出借位
P_i, G_i	超前进位中的传递、产生函数; 本讲义 $P_i = a_i \oplus b_i$
CE, EN, LD	时钟使能、一般使能、装载控制; 有效极性须单独规定
下标 n 与小圆圈	常用来标识低有效控制, 但最终以功能表为准
RCO	进位/终值或级联信号, 波形定义依器件
X, Z 与 x, z	大写可作普通变量名; HDL 小写值分别为未知、高阻
RTL、FSM	寄存器传输级、有限状态机
SIPO、PISO	串入并出、并入串出
PLD、CPLD、FPGA	可编程逻辑器件、复杂 PLD、现场可编程门阵列

22.3 知识联系与易混概念

数制确定数量的解释, 编码确定对象与码字的对应; 真值表确定组合函数, 标准式保证描述完整, 卡诺图与布尔代数减少实现项数; 译码器产生最小项, MUX 实现按选择条件展开, ROM/LUT 直接存储真值表。

时序电路在组合逻辑之外增加状态存储。触发器特征方程把驱动函数变成次态函数, 激励表把期望次态变成可实现的驱动。寄存器强调保存和传送, 计数器强调固定状态循环, 状态机强调受输入控制的状态变化。序列检测处理输入历史, 序列发生器按内部状态产生输出。

逻辑真值相同与瞬态波形相同须分开; 同步计数与同步清零须分开; 使能保持与禁用清零须分开; 驱动高阻与主动驱动零须分开; 化简无关项与仿真未知须分开。状态可自启动不代表初相位固定, 也不保证恢复期间的输出正确。

22.4 实验主要结论与代码术语

wire用于连接, reg用于过程赋值; 边沿过程表示时序状态, 完整always @*表示组合规则; =与<=必须按求值时序解释。连续稳定过滤解决机械抖动, 同步链降低异步输入的亚稳态传播风险, 边沿检测把稳定电平变为单次事件, 三者作用不同。

从 0 计数的长度为终值加一。移位频率、完整 LED 循环频率、方波翻转频率和完整亮灭频率不可混用。tick通常是原时钟域的单拍使能。扫描索引决定数码位置, 译码表决定段形状, 有效电平由具体板卡规定。

UART 一帧的八位数据最低位先传，ASCII 字符与十六进制数值需区分。valid 表示当前边沿有一份可接受的数据，不能当作持续请求。发送忙碌反馈、接收停止位验证、末两位计数以及字符串终结符，决定连续运行的边界行为。状态代表已接受输入的历史，输入无效时保持状态。

22.5 本次实验验证记录与边界

整理示例使用 Icarus Verilog 进行自动功能仿真：四位加减器 1024 组、译码器 64 组、两种寄存器文件八个地址及异步复位保持、Moore 与 Mealy 检测器各 256 个字、格雷循环 32 次转移、四拍节拍、短毛刺拒绝与长按单次事件、模 100 计数 256 次；改写 UART 逐字节验证全部 256 种发送数据和全部 256 种接收数据，并验证短起始毛刺、坏停止位拒绝及随后恢复。

原 UART 发送测试五组数据、接收测试十组数据均通过。原测试程序以byte命名任务参数，使用传统 Verilog 模式可运行，若按 SystemVerilog 编译则与关键字冲突；不能把这一语言模式错误当成 UART 逻辑错误。原 Lab 4、5、6 的顶层完成语法与展开检查。

额外定向仿真确认原接收显示累计 256 个有效字符后显示 00，而低两位应为 56；成功匹配start后收到CRLF会提出31与30两次回复请求，后者为额外零回复。原始文件未改写，这些问题在正文和配套说明中标出。整理示例的验证只涉及所列模块和范围；未在 Vivado 中综合、布局布线或上板，不宣称全部参考工程已通过硬件验收。2020 年记忆游戏为需求与设计解析，没有原始完整 RTL 可验证。

22.6 课件范围与资料说明

新增例题参考《2024 秋数逻期末回忆版》（资料标注更新日期 2024 年 11 月 25 日）与上述课件。电路图按可辨认接线重绘，逐项列出输入条件、位序、有效电平和初态；改编题注明补充条件。地址译码题的位序与通常 74LS138 约定冲突，正文分别给出两种解释；波形设计题重新规定明确的周期与初态。回忆资料未能确认的细节不据此补写为原题条件。

本讲义以桌面“数字逻辑设计课件”文件夹内全部 15 份 PDF 为主要依据。重复版本合并为 13 个课件主题，保留较长版本中的补充内容，按知识联系整合为前述章节。课件中的教学安排、考核比例、建议和课堂投票不作为学科正文重复收录。

课件标识	内容	对应正文
01 引言	基本概念、数字系统、设计层次	第 1 章
02 数制和编码	位权、进制转换、BCD、余 3、格雷、字符编码	第 2 章
03-1 逻辑代数	逻辑门、真值表、定律、反演、对偶、代数化简	第 3 章
03-2 布尔代数应用	标准式、最大最小项、不完全给定函数	第 3、4 章

课件标识	内容	对应正文
03-3 卡诺图	多变量图、读式、无关项、代码转换	第 4、5 章
03-4 险象及消除	延迟、冒险、判别与消除	第 6 章
04-1 组合逻辑设计	表决、运算、加减器、三态门	第 7 章
04-2 常用组合器件	译码、MUX、编码、校验、比较、ROM、PLD	第 8、9 章
05 Verilog HDL	模块、数据类型、运算、赋值、过程、仿真	第 9 章
06-1 时序元件	锁存器、触发器、附加控制、类型转换	第 10 章
06-2 寄存器与计数器	寄存器、移位、计数、节拍、HDL	第 11 章
06-03 同步时序设计	状态图、状态化简、编码、序列检测	第 12 章
06-04 中规模芯片应用	计数器、级联、寄存器、序列发生器	第 13 章

04-2 的 65 页与 68 页版本合并，后者补充的优先编码、奇偶校验及分组比较 Verilog 应用均纳入对应主题。06-2 的两个 47 页版本提取内容一致，仅保留一套讲解。讲义中的小规模算术、码制转换、反馈状态及序列例子按定义逐值核对；硬件延迟与非法控制条件未用理想真值表替代。

必要纠正包括：标准 ASCII 为七位；十个数码从 16 个码字中任选一组共有 $\binom{16}{10} = 8008$ 种选集，而给十个数码分别分配不同码字有 $16!/6!$ 种映射，二者不同；锁存器可带异步复位，完整赋值与过程类型决定是否推断锁存器；FSM 不必有终态；状态位下界须使用向上取整；同步计数的组合译码仍可能有毛刺；74LS193 清零高有效、74LS190/191 级联端功能不同；若未给出适用统计模型，不将“单比特错误占 96%”当作通用概率。

外部核验仅用于澄清标准、器件极性和明显错误，未扩展课件学科范围。可核对前文链接的 RFC 与 TI 原厂资料，以及 Intel 寄存器与锁存器编码指南。器件图引脚名称、端口次序、有效电平和允许同时控制条件应按具体型号的数据手册执行。

22.7 实验资料与代码归档

实验资料读取时间为 2026 年 10 月 4 日。2025 年六次实验的要求来自课程页面链接仓库中各次“实验指导书”PDF，设计来自“源代码”目录；报告只用于辅助核对，不用报告结论替代 RTL 与功能表。2020 年指导书与 2024 年补充取自 HOA 课程仓库 labs。配套目录记录原始下载路径及 SHA-256 摘要，可区分原文件与规范化示例。

后续资料到来时，先确定实验版本、开发板、接口及任务变化，再在本讲义最新源文件中更新相应章节和验证记录，保留已有理论内容及统一样式。不能把另一学年的题目直接覆盖成同名实验的唯一要求。